

OFICINA VIRTUAL

FAROUK CHAJRI KASBAJ
IEES CARMEN MARTÍN GAITE
2024-2025

CFGS DESARROLLO DE APLICACIONES MULTIPLATAFORMA

ÍNDICE

1	Dedicatorias y agradecimientos	4
2	Introducción	5
3	Objetivos del proyecto	6
3.1	Requisitos del Sistema	6
3.1.1	Requisitos funcionales.....	6
3.1.2	Requisitos técnicos	7
3.1.3	Requisitos Mínimos del PC para Ejecutar la Aplicación	7
3.1.4	Requisitos mínimos:.....	7
3.1.5	Requisitos recomendados:.....	8
4	Marco conceptual y contextual	8
4.1	Contexto del Proyecto	8
4.2	Estado del Arte y Tecnologías Comparables	9
4.3	Técnicas y Tecnologías Utilizadas	9
4.3.1	Unity 2D (2022.3.60f1 LTS)	10
4.3.2	Mirror (Framework de red para Unity)	12
4.3.3	Vivox (Chat de voz en tiempo real)	12
4.3.4	Backend en PHP + NGINX + MySQL (sobre AWS EC2).....	14
4.3.5	Amazon S3 (Almacenamiento cifrado en la nube con SSE-S3)	16
4.3.6	Seguridad: HTTPS, Tokens y Cifrado.....	16
4.3.7	Conclusión.....	16
5	Metodología y organización.....	17
5.1	Planificación.....	17
5.2	Diagrama de Gantt	17
5.3	Análisis DAFO.....	17
5.4	Viabilidad del Proyecto	18
5.5	Cronograma de actividades:	19
6	Desarrollo	19
6.1	Fases del desarrollo real y comparación con la planificación	20
6.2	Resultados obtenidos.....	22
6.3	Diagramas UML.....	23
6.3.1	Diagrama de caso de uso	23
6.3.2	Diagrama de clases.....	23

6.3.3	Diagrama de actividades	24
6.3.4	Diagrama de secuencia.....	25
6.3.5	Diagrama de componente.....	26
7	Plan de pruebas.....	26
7.1	Metodología de prueba.....	26
7.2	Pruebas funcionales.....	27
7.3	Pruebas de rendimiento	28
7.4	Pruebas de seguridad.....	28
7.5	Pruebas de usabilidad	29
8	Resultado y conclusiones y vías de mejora.....	29
8.1	Resultados obtenidos.....	29
8.2	Conclusiones	30
8.3	Vías de mejora.....	30
9	Anexos.....	31
9.1	Documentos de planificación	31
9.1.1	Plan de hitos	31
9.1.2	Diagramas de Gantt.....	32
9.1.3	Cronograma de actividades:	32
9.2	Manual de uso	38
9.2.1	Acceso al programa.....	38
9.2.2	Entorno principal.....	39
9.2.3	Funciones del menú lateral.....	41
9.2.4	Navegación entre escenas.....	44
10	Bibliografía.....	46

1 DEDICATORIAS Y AGRADECIMIENTOS

Este trabajo está dedicado a todas las personas que han sido parte de mi camino académico y personal. A mis familiares, por su apoyo incondicional y por creer siempre en mí, incluso en los momentos de incertidumbre. A mis amigos, por su motivación y por estar a mi lado, compartiendo risas y consejos cuando más los necesitaba. A mis profesores, por su orientación y enseñanza, que me han permitido llegar hasta aquí. Y, especialmente, a mí mismo, por no rendirme y comprometerme conmigo mismo para seguir adelante sin importar las circunstancias.

Quisiera expresar mi más sincero agradecimiento a todas las personas que han contribuido, directa o indirectamente, en la realización de este proyecto.

En primer lugar, agradezco a mi tutor, Sergio Urbano Ruiz, por su apoyo y orientación a lo largo de todo el proceso. Su conocimiento y paciencia han sido fundamentales para llevar este proyecto a buen puerto.

Agradezco a todos los profesores y compañeros que, de alguna manera, han enriquecido mi aprendizaje durante mi carrera y me han proporcionado las herramientas necesarias para desarrollar este trabajo.

No puedo dejar de agradecer a mi familia, quienes siempre me han apoyado en cada paso, desde las dificultades hasta los momentos de alegría. Su confianza ha sido mi mayor impulso.

A mis amigos, por ofrecerme un respiro cuando más lo necesitaba, por sus palabras de aliento y por su compañía. Vuestra amistad ha sido esencial para no perder el rumbo durante este intenso proceso.

Y, finalmente, a todas aquellas personas que, de alguna forma, han influido en este proyecto. Su contribución, aunque a veces silenciosa, ha dejado una huella en el desarrollo de este trabajo.

2 INTRODUCCIÓN

En un mundo cada vez más interconectado y digitalizado, el trabajo remoto ha adquirido una relevancia sin precedentes. El presente Proyecto de Fin de curso (PFC) tiene como objetivo el desarrollo de una **oficina virtual interactiva**, un entorno digital diseñado para facilitar la colaboración profesional a distancia, combinando herramientas de comunicación en tiempo real con una experiencia visual similar a entornos lúdicos como *Habbo Hotel*. Sin embargo, a diferencia de este último, nuestro proyecto tiene una orientación puramente profesional, enfocada en la eficiencia y la seguridad.

La motivación principal surge de la necesidad de contar con espacios virtuales más humanos, donde los usuarios no solo puedan intercambiar archivos o comunicarse por texto, sino también sentir una mayor presencia y cercanía con sus compañeros, como si compartieran un entorno físico. Para ello, se ha diseñado una plataforma en la que los avatares pueden desplazarse libremente en un entorno 2D, interactuar mediante chat de voz con detección de proximidad, compartir archivos cifrados de forma segura (mediante tecnologías AES y RSA), e incluso visualizar pizarras colaborativas o zonas de reunión.

El proyecto resulta especialmente interesante por su carácter transversal, ya que combina aspectos de desarrollo de videojuegos (Unity, animación, sincronización de avatares), con programación de sistemas distribuidos (servidores, bases de datos, WebSockets, PHP), y ciberseguridad (cifrado, autenticación mediante tokens). Asimismo, el uso de tecnologías modernas como *AWS S3*, *Vivox* para chat de voz, y *Mirror* para sincronización en red, permite al estudiante enfrentarse a desafíos reales del desarrollo de software profesional.

Además, este PFC no solo se centra en el desarrollo técnico, sino también en la reflexión crítica sobre la arquitectura, la escalabilidad y la seguridad del sistema. El trabajo documenta errores, decisiones y correcciones realizadas durante el proceso, evidenciando un aprendizaje continuo y una capacidad de adaptación fundamental para cualquier desarrollador de software.

En resumen, este proyecto representa una aportación práctica y significativa al campo del trabajo colaborativo online, explorando cómo la virtualización de espacios laborales puede evolucionar más allá de las herramientas convencionales como Zoom o Google Meet, ofreciendo una experiencia más inmersiva, eficiente y segura.

3 OBJETIVOS DEL PROYECTO

El objetivo principal del proyecto es el desarrollo de una **oficina virtual interactiva** que permita a varios usuarios conectarse simultáneamente a un entorno visual 2D, con funcionalidades orientadas al trabajo colaborativo remoto, la comunicación en tiempo real y la seguridad en la transmisión de datos.

Los objetivos específicos del proyecto son:

- **Sincronización de usuarios en tiempo real:** Implementar un sistema de movimiento de avatares que refleje con baja latencia la posición y animación de cada usuario.
- **Comunicación multimedia:** Integrar un sistema de chat de texto y voz con detección de proximidad, utilizando tecnologías como Mirror (para red) y Vovox (para voz).
- **Transferencia segura de archivos:** Desarrollar una interfaz para el envío de archivos cifrados mediante PHP, almacenados en la nube (AWS S3) y protegidos con cifrado del lado del servidor (SSE-S3).
- **Gestión de sesiones y presencia:** Crear tablas en la base de datos para rastrear usuarios conectados, salas activas y archivos pendientes por recibir.
- **Accesibilidad y experiencia de usuario:** Diseñar una interfaz atractiva, funcional y adaptable a distintas resoluciones de pantalla.
- **Escalabilidad y mantenimiento:** Documentar las buenas prácticas aplicadas, los errores corregidos y las decisiones técnicas tomadas, para facilitar futuras ampliaciones o mantenimiento del sistema.
- **Compartir pantalla:** Integrar un sistema que permita a los usuarios compartir pantalla dentro de la aplicación.

3.1 Requisitos del Sistema

3.1.1 Requisitos funcionales

- Registro, inicio de sesión y recuperación de cuenta mediante autenticación con base de datos remota.
- Movimiento de avatares y sincronización entre clientes mediante Mirror.
- Chat de texto y voz con canales configurables y comunicación por proximidad.
- Subida y descarga de archivos entre usuarios, con cifrado de extremo a extremo.
- Gestión de notificaciones, presencia online y visualización de usuarios en tiempo real.
- Paneles interactivos para crear salas de voz, enviar archivos y recibir archivos.

3.1.2 Requisitos técnicos

- **Frontend:** Unity 2022.3.60f1 LTS (2D).
- **Backend:** Servidor en Ubuntu 22.04 (AWS EC2), .NET Core, PHP, NGINX.
- **Red:** Mirror para sincronización de clientes, WebSockets, Vivox para voz.
- **Base de Datos:** PostgreSQL (AWS RDS), gestionada con PHP.
- **Almacenamiento en la nube:** AWS S3 con cifrado SSE-S3.
- **Seguridad:** Autenticación mediante tokens, cifrado de archivos en servidor, protección básica contra accesos no autorizados.

Este conjunto de objetivos y requisitos garantiza que el proyecto se mantenga alineado con los estándares académicos del PFC, al mismo tiempo que persigue una solución funcional, segura y extensible para entornos colaborativos.

3.1.3 Requisitos Mínimos del PC para Ejecutar la Aplicación

Dado que la oficina virtual está desarrollada en Unity 2D y utiliza gráficos simples tipo píxel art, los requisitos técnicos para el usuario final son bastante modestos. A continuación, se presentan las especificaciones mínimas y recomendadas para un funcionamiento correcto:

3.1.4 Requisitos mínimos:

- **Sistema operativo:** Windows 10
- **Procesador:** Intel Core i3 (2 núcleos) o equivalente
- **Memoria RAM:** 4 GB
- **Tarjeta gráfica:** Integrada (Intel HD Graphics o equivalente)
- **Almacenamiento:** 300 MB de espacio libre
- **Resolución de pantalla:** 1024x768
- **Conexión a Internet:** 1 Mbps de bajada/subida (estable)
- **Auriculares y micrófono:** Opcional, solo necesarios para el chat de voz

3.1.5 Requisitos recomendados:

- **Sistema operativo:** Windows 10 o Superior
- **Procesador:** Intel Core i5 o AMD equivalente
- **Memoria RAM:** 8 GB
- **Almacenamiento:** SSD con al menos 500 MB libres
- **Conexión a Internet:** 5 Mbps o superior para una experiencia de voz más fluida

Este rango de requisitos permite que la aplicación funcione correctamente en la mayoría de los ordenadores personales actuales, incluso en equipos modestos, facilitando su uso por parte de usuarios sin hardware especializado.

4 MARCO CONCEPTUAL Y CONTEXTUAL

El presente proyecto se sitúa en la intersección entre el trabajo colaborativo remoto, el desarrollo de videojuegos 2D, y la comunicación multimedia en tiempo real. Esta combinación da lugar a una oficina virtual interactiva que proporciona una experiencia de usuario más inmersiva que las plataformas tradicionales, como Zoom o Google Meet, al incorporar un entorno gráfico animado donde los usuarios pueden “desplazarse” y relacionarse visualmente.

4.1 Contexto del Proyecto

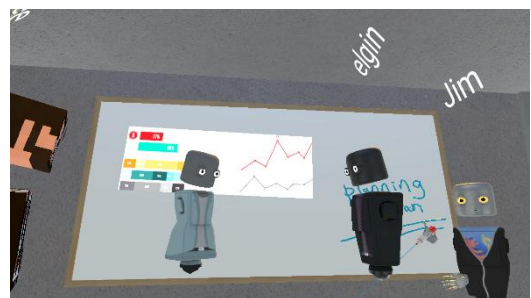
Desde la pandemia de COVID-19, el teletrabajo ha evolucionado de ser una opción puntual para convertirse en una modalidad habitual en muchas empresas. Sin embargo, las herramientas convencionales de trabajo remoto se han centrado principalmente en videollamadas, mensajería y almacenamiento en la nube, dejando de lado aspectos como la presencia espacial o la interacción visual no verbal.

Frente a esta limitación, han surgido iniciativas como **Gather**, **Teamflow** o **Remo**, que simulan entornos virtuales donde los participantes pueden moverse con avatares por oficinas digitales. Estas plataformas buscan hacer más “humana” la experiencia del trabajo remoto. No obstante, suelen depender de navegadores web, carecen de personalización profunda, o implican costes elevados.

4.2 Estado del Arte y Tecnologías Comparables

A nivel técnico y funcional, el proyecto toma inspiración de plataformas como:

- **Habbo Hotel:** juego social en 2D que permite a los usuarios moverse, interactuar y comunicarse mediante chat. Su enfoque es más lúdico que profesional.
- **Gather.town:** plataforma de videollamadas en entorno 2D con avatares y salas interactivas. Requiere navegador moderno, pero no permite transferencia cifrada de archivos ni integración profunda con servidores propios.
- **Mozilla Hubs:** experiencia de realidad virtual colaborativa en 3D, enfocada más en entornos educativos y de investigación.



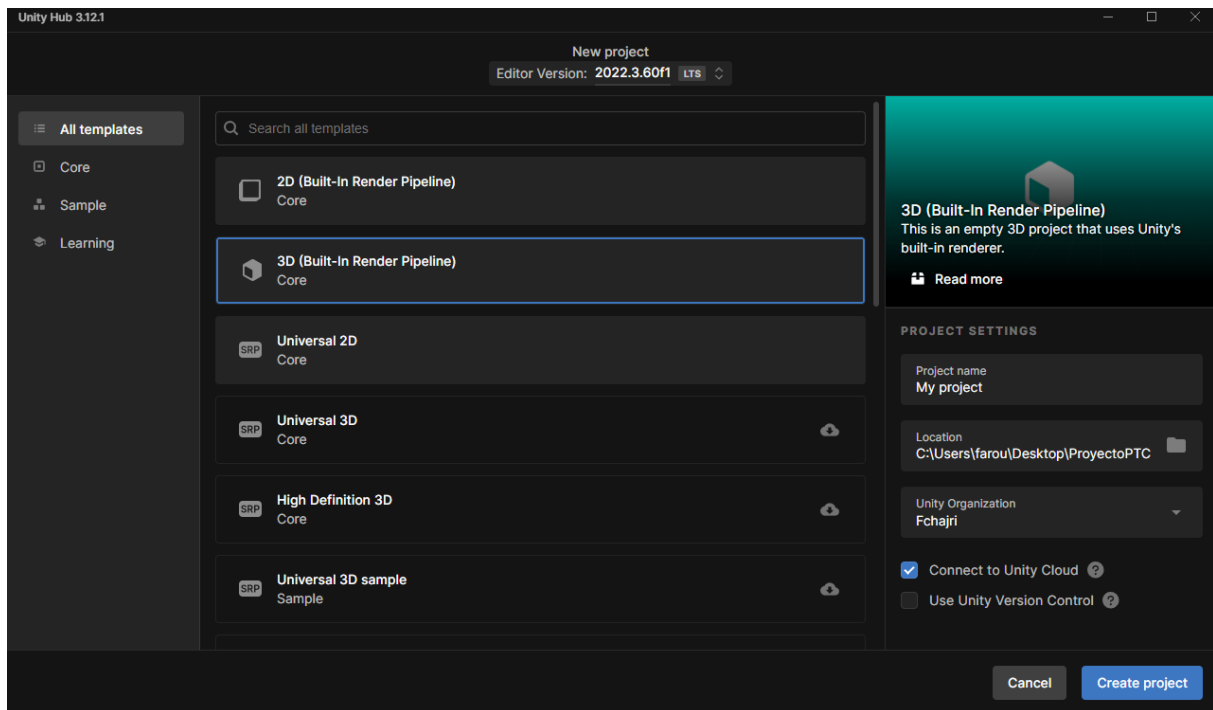
Este proyecto busca cubrir una necesidad intermedia: ofrecer un entorno ligero, 2D, profesional, personalizable y seguro, que funcione sin navegador web y que permita mayor control sobre los datos y el entorno.

4.3 Técnicas y Tecnologías Utilizadas

El desarrollo del proyecto ha requerido el uso coordinado de múltiples tecnologías actuales, cada una especializada en una parte clave del sistema: motor gráfico, comunicación en red, backend, almacenamiento, seguridad y comunicación en tiempo real. A continuación, se detallan dichas tecnologías, junto con su integración en el proyecto y su relevancia actual en la industria.

4.3.1 Unity 2D (2022.3.60f1 LTS)

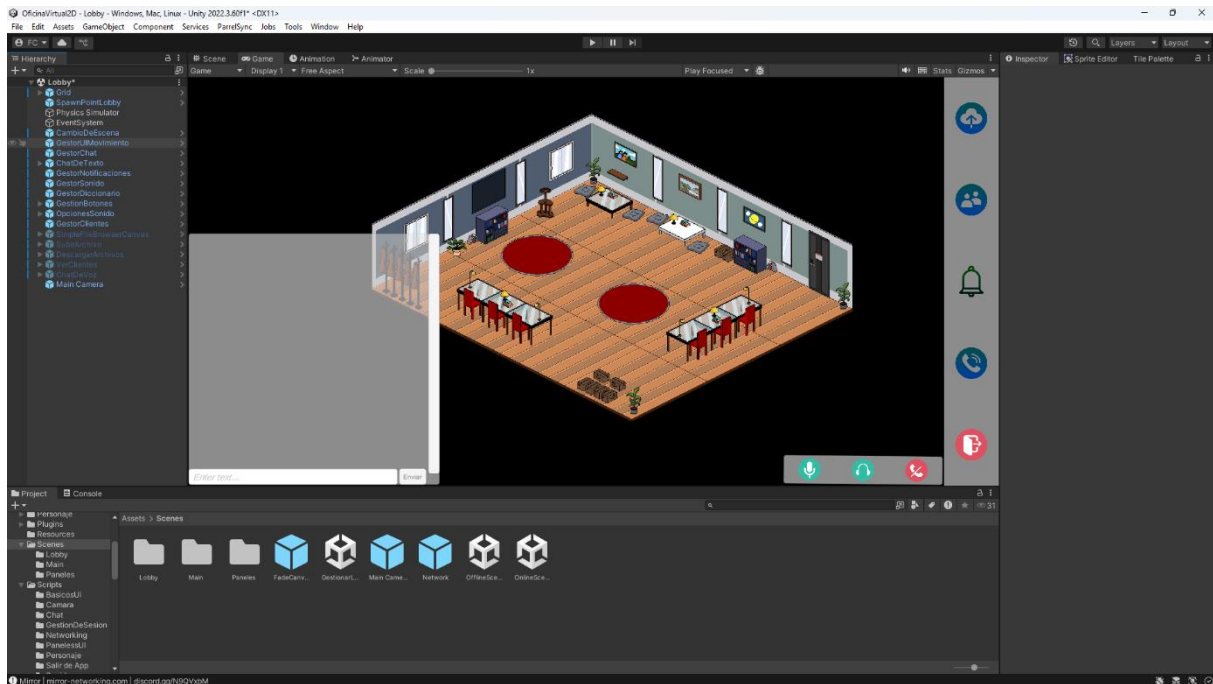
Unity es un motor de desarrollo multiplataforma utilizado para crear videojuegos, simulaciones interactivas y aplicaciones 2D/3D. Es uno de los entornos más populares en la industria por su facilidad de uso, versatilidad y gran comunidad de desarrolladores. Permite diseñar visualmente escenas, programar comportamientos, gestionar interfaces gráficas, controlar animaciones, físicas, y mucho más.



HUD de Unity para crear proyectos.

Unity sigue un enfoque basado en componentes: cada objeto dentro del juego (como un personaje, una puerta o una pared) puede tener distintos "componentes" como físicas, animaciones, scripts, colisiones, etc., que definen su comportamiento. Esto permite una programación modular, clara y muy adaptable.

En este proyecto, se utilizó Unity **en su modo 2D**, que está especialmente orientado a proyectos con gráficos planos (sin profundidad). Todo el entorno —el lobby, la oficina, los menús, y los personajes— se ha diseñado en este entorno visual. Los gráficos fueron obtenidos a través de bibliotecas de recursos (assets), adaptados a una cuadrícula para facilitar el movimiento y la coherencia espacial del entorno.

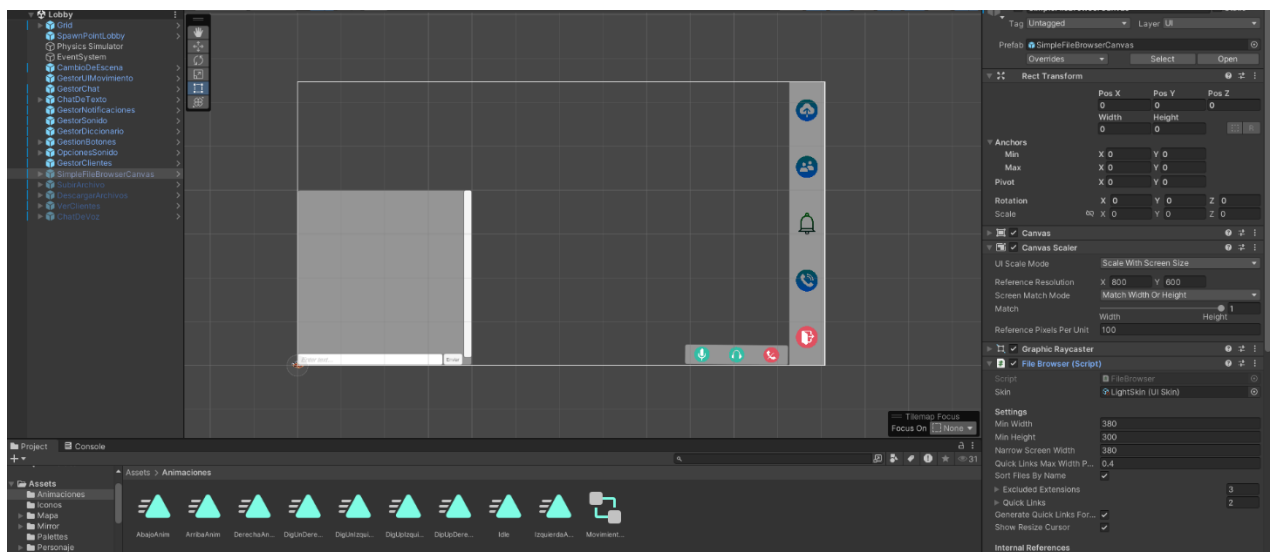


Interfaz de Unity y zona de trabajo.

Unity también facilita la creación de interfaces gráficas de usuario (UI), lo que permitió implementar menús para el inicio de sesión, registro, gestión de archivos, notificaciones, y paneles de chat. Además, Unity sirve como el cliente principal que se conecta con el servidor para intercambiar información en tiempo real.

Ejemplos reales de uso:

Juegos ampliamente conocidos como **Hollow Knight**, **Cuphead**, **Ori and the Blind Forest**, y **Among Us** fueron desarrollados en Unity. También se utiliza para crear experiencias interactivas en entornos educativos, arquitectura, medicina y simuladores.



Interfaz de Unity

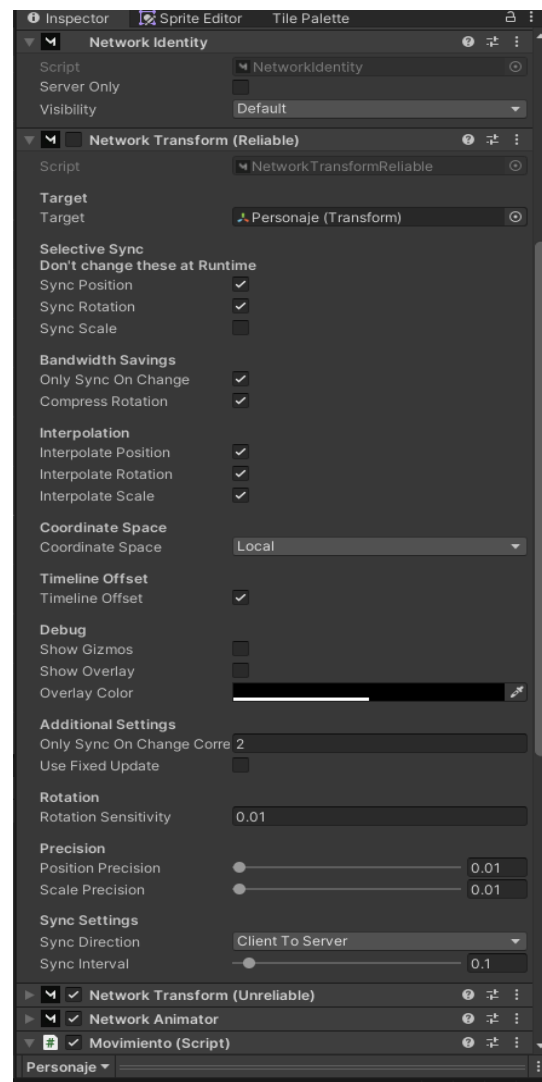
4.3.2 Mirror (Framework de red para Unity)

Mirror es un sistema de red de código abierto creado como sucesor optimizado de UNET (el sistema anterior de Unity). Es uno de los frameworks de red más usados actualmente en proyectos independientes gracias a su simplicidad, extensibilidad y buen rendimiento.

Se utiliza para sincronizar el movimiento de los avatares, gestionar la conexión entre clientes y servidor, y permitir el funcionamiento del chat de texto. Dado que el ejemplo base contenía malas prácticas, se reescribió parte del código para mejorar rendimiento y claridad.

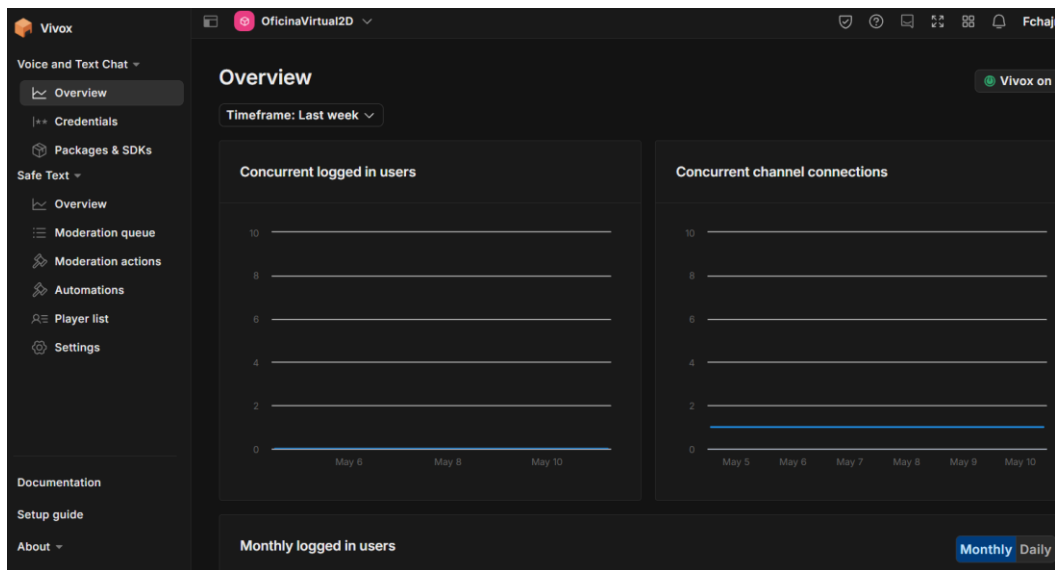
La imagen contiene elementos de Mirror necesarios para el funcionamiento del personaje con la red tales como Network Identity, transform etc.

Ejemplos reales de uso: Juegos como Population: One, Eco y algunos títulos en Steam Greenlight utilizan Mirror como base de red.



4.3.3 Vivox (Chat de voz en tiempo real)

Vivox es uno de los sistemas de chat de voz más utilizados en la industria del videojuego profesional. Fue adquirido por Unity y está integrado gratuitamente para desarrolladores con proyectos registrados. Ofrece audio posicional, gestión de canales y excelente calidad de voz.



Panel de Vivox en Unity cloud

En este proyecto, se integró el SDK de Vivox para permitir salas de voz como “Lobby”, “Oficina” y canales privados. El proceso fue complejo debido a problemas de configuración entre el build server y las librerías, pero se solucionó tras varios días de depuración.



Interfaz de Unity donde se usa Vivox.

Ejemplos reales de uso: Fortnite, Valorant, League of Legends y PUBG usan Vivox como su sistema de comunicación principal.

4.3.4 Backend en PHP + NGINX + MySQL (sobre AWS EC2)

Para el funcionamiento del servidor, se configuró una instancia EC2 (una máquina virtual) en Amazon Web Services (AWS), ejecutando Ubuntu como sistema operativo. Esta instancia actúa como el núcleo del backend, es decir, la parte del sistema que gestiona los datos, usuarios, archivos y la lógica fuera del juego.

El backend se compone de tres tecnologías principales:

- **NGINX** es un servidor web muy utilizado por su velocidad y bajo consumo de recursos. Su función principal es recibir las peticiones que llegan desde los clientes (jugadores) y redirigirlas a los scripts PHP que procesan la lógica.
- **PHP** es un lenguaje de programación del lado del servidor, diseñado especialmente para crear páginas y servicios web. En este proyecto, PHP se utilizó para implementar una **API propia** que permite registrar usuarios, iniciar sesión, enviar archivos, recibirlos, y verificar notificaciones. Su facilidad de despliegue en entornos Linux lo hace ideal para proyectos con infraestructura controlada.
- **MySQL** es un sistema de gestión de bases de datos relacional (RDBMS) que permite almacenar, consultar y organizar grandes cantidades de información estructurada. En este caso, se utiliza para guardar datos de usuarios, contraseñas cifradas, archivos enviados, notificaciones y actividad dentro de la oficina virtual.

Además, se protegieron todas las rutas sensibles del backend mediante **tokens de seguridad**, lo que impide que usuarios externos puedan acceder directamente a los scripts desde fuera del cliente autorizado.

A continuación, se muestran imágenes de donde están alojados dichos archivos:

```
drwxrwxr-x.  4 ec2-user ec2-user 16384 Apr 27 13:05 .
drwxr-xr-x.  4 root     root      33 Apr 10 10:59 ..
-rwxrwxr-x.  1 ec2-user ec2-user  3650 Feb 11 01:59 404.html
-rwxrwxr-x.  1 ec2-user ec2-user  3693 Feb 11 01:59 50x.html
-rw-r--r--.  1 ec2-user ec2-user  1651 Apr 16 13:06 changePassword.php
-rw-r--r--.  1 ec2-user ec2-user  1586 Apr 17 15:57 checkUser.php
-rw-r--r--.  1 ec2-user ec2-user    63 Apr 17 13:23 composer.json
-rw-r--r--.  1 ec2-user ec2-user 32664 Apr 17 13:23 composer.lock
-rw-r--r--.  1 ec2-user ec2-user  1953 Apr 21 12:24 downloadDocs.php
-rw-r--r--.  1 ec2-user ec2-user  3541 Apr 27 13:35 entradaSalidaChatDeVoz.php
-rw-r--r--.  1 ec2-user ec2-user  2183 Apr 19 15:01 getNotification.php
drwxrwxr-x.  2 ec2-user ec2-user    27 Apr 10 10:59 icons
-rwxrwxr-x.  1 ec2-user ec2-user   615 Feb 11 02:00 index.html
-rw-r--r--.  1 ec2-user ec2-user  1718 Apr 16 12:55 login.php
-rwxrwxr-x.  1 ec2-user ec2-user   368 Feb 11 01:59 nginx-logo.png
-rw-r--r--.  1 ec2-user ec2-user  1335 Apr 27 14:17 obtenerUsuariosEnSala.php
lrwxrwxrwx.  1 ec2-user ec2-user    14 Feb 11 02:00 poweredby.png -> nginx-logo.png
-rw-r--r--.  1 ec2-user ec2-user  1773 Apr 16 13:06 recoverAccount.php
-rw-r--r--.  1 ec2-user ec2-user  2202 Apr 16 13:04 registrarse.php
-rw-r--r--.  1 ec2-user ec2-user  3436 Apr 21 10:39 uploadNotification.php
-rw-r--r--.  1 ec2-user ec2-user  1549 Apr 21 12:14 uploader.php
-rw-r--r--.  1 ec2-user ec2-user  1245 Apr 20 15:40 userName.php
drwxr-xr-x. 10 ec2-user ec2-user   142 Apr 17 13:21 vendor
```

/usr/share/nginx/html/				
Nombre	Tamaño	Modificado	Permisos	Propieta.
.		10/04/2025 10:59:22	rw-r--r--	root
icons		10/04/2025 10:59:22	rw-rw-r--	ec2-user
vendor		17/04/2025 13:21:35	rw-r--r--	ec2-user
50x.html	4 KB	11/02/2025 1:59:31	rw-rw-r--	ec2-user
404.html	4 KB	11/02/2025 1:59:31	rw-rw-r--	ec2-user
changePassword.php	2 KB	16/04/2025 13:06:27	rw-r--r--	ec2-user
checkUser.php	2 KB	17/04/2025 15:57:00	rw-r--r--	ec2-user
composer.json	1 KB	17/04/2025 13:23:17	rw-r--r--	ec2-user
composer.lock	32 KB	17/04/2025 13:23:17	rw-r--r--	ec2-user
downloadDocs.php	2 KB	21/04/2025 12:24:19	rw-r--r--	ec2-user
entradaSalidaChatDeV...	4 KB	27/04/2025 13:35:17	rw-r--r--	ec2-user
getNotification.php	3 KB	19/04/2025 15:01:37	rw-r--r--	ec2-user
index.html	1 KB	11/02/2025 2:00:47	rw-rw-r--	ec2-user
login.php	2 KB	16/04/2025 12:55:43	rw-r--r--	ec2-user
nginx-logo.png	1 KB	11/02/2025 1:59:31	rw-rw-r--	ec2-user
obtenerUsuariosEnSala...	2 KB	27/04/2025 14:17:18	rw-r--r--	ec2-user
poweredby.png	1 KB	11/02/2025 2:00:47	rw-rw-rw-r	ec2-user
recoverAccount.php	2 KB	16/04/2025 13:06:29	rw-r--r--	ec2-user
registrarse.php	3 KB	16/04/2025 13:04:52	rw-r--r--	ec2-user
uploader.php	2 KB	21/04/2025 12:14:50	rw-r--r--	ec2-user
uploadNotification.php	4 KB	21/04/2025 10:39:52	rw-r--r--	ec2-user
userName.php	2 KB	20/04/2025 15:40:13	rw-r--r--	ec2-user

Ejemplos reales de uso:

- **PHP** es utilizado por grandes plataformas como WordPress (el CMS más usado del mundo), Wikipedia, y sitios desarrollados con Laravel.
- **NGINX** es utilizado por sitios de alto tráfico como Netflix, Dropbox o GitHub por su eficiencia como servidor web.
- **MySQL** es utilizado por Facebook, PayPal, YouTube y miles de servicios web por su fiabilidad y rendimiento.

4.3.5 Amazon S3 (Almacenamiento cifrado en la nube con SSE-S3)

Amazon S3 permite almacenar archivos en la nube con escalabilidad y alta disponibilidad. En este proyecto, los archivos enviados por los usuarios se almacenan en un bucket configurado para cifrar automáticamente todos los datos con SSE-S3 (Server-Side Encryption).

cloudoficinavirtual Información

Objetos | Propiedades | Permisos | Métricas | Administración | Puntos de acceso

Objetos (5) Copiar URI de S3 Copiar URL Descargar Abrir Eliminar

Los objetos son las entidades fundamentales que se almacenan en Amazon S3. Puede utilizar el [inventario de Amazon S3](#) para obtener una lista de todos los objetos de su bucket, que concederles permisos de forma explícita. [Más información](#)

Buscar objetos por prefijo

☐	Nombre	Tipo	Última modificación	Tamaño	Clase de almacenamiento
☐	680637297e6f1_imagen.png	png	21 Apr 2025 2:16:42 PM CEST	16.1 KB	Estándar
☐	680647194ab56_imagen.png	png	21 Apr 2025 3:24:42 PM CEST	16.1 KB	Estándar
☐	68091432b9bb5_cloud_A.xlsx	xlsx	23 Apr 2025 6:24:19 PM CEST	12.5 KB	Estándar
☐	681758ea63c49_Auri.png	png	4 May 2025 2:09:15 PM CEST	21.4 KB	Estándar
☐	681ddbe35059e_auriculares.png	png	9 May 2025 12:41:40 PM CEST	22.8 KB	Estándar

Imagen de los datos almacenados en Amazon S3

Además, se utilizó IAM (Identity and Access Management) para controlar los permisos del bucket, evitando exponer credenciales dentro del código del cliente.

Ejemplos reales: Netflix, Dropbox y Pinterest usan Amazon S3 para almacenamiento de archivos y contenido multimedia.

4.3.6 Seguridad: HTTPS, Tokens y Cifrado

- Se adquirió un dominio propio para poder habilitar HTTPS con un certificado SSL válido.
- Se implementó un sistema de tokens únicos para validar las operaciones del cliente.
- La subida y descarga de archivos se hace a través de endpoints seguros.
- AWS S3 se encarga del cifrado automático (sin intervención del desarrollador).

Estas medidas fueron esenciales para simular un entorno profesional y seguro, tal como sería necesario en una aplicación empresarial real.

4.3.7 Conclusión

La solución desarrollada combina estas tecnologías de manera personalizada, creando una plataforma única que permite moverse con un avatar, comunicarse por voz o texto, enviar archivos cifrados y ver a otros usuarios en tiempo real, todo dentro de una única aplicación de escritorio ligera y multiplataforma.

Además, se ha puesto especial énfasis en la eficiencia, la modularidad del código, y la posibilidad de ampliarlo o integrarlo con otros servicios, lo que convierte este proyecto no solo en una solución funcional, sino también en una base extensible para futuros desarrollos.

5 METODOLOGÍA Y ORGANIZACIÓN

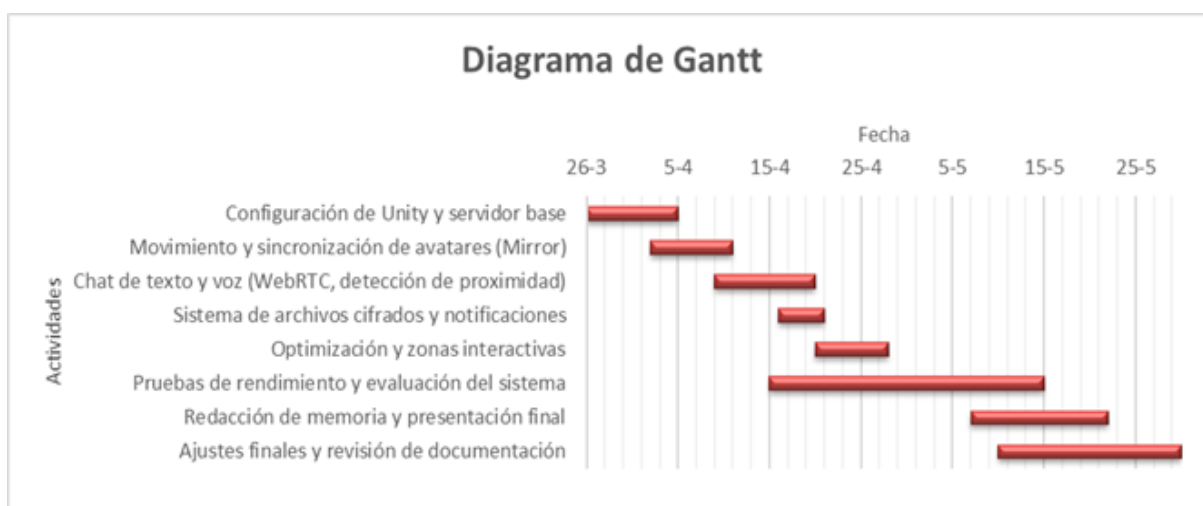
5.1 Planificación

La planificación inicial del proyecto contemplaba un desarrollo de 7 semanas de trabajo activo, con jornadas de entre 5 y 6 horas diarias, más 2 semanas adicionales para pruebas finales, ajustes y documentación. A lo largo del proyecto, se ajustaron varias tareas frente a imprevistos técnicos, como los problemas con el chat de texto, la integración de Vivox o la migración de Apache a Nginx. A pesar de ello, se mantuvo el calendario general gracias a la flexibilidad en la planificación.

5.2 Diagrama de Gantt

Este diagrama de Gantt refleja la planificación inicial del proyecto "Oficina Virtual Interactiva".

A modo de referencia, en el apartado [9.1.2](#) se presenta el diagrama de Gantt correspondiente al desarrollo real del proyecto, en el que se pueden observar las diferencias entre la planificación prevista y la ejecución efectiva de las tareas.



5.3 Análisis DAFO

El siguiente análisis DAFO identifica las Fortalezas, Debilidades, Oportunidades y Amenazas del proyecto:

Fortalezas: - Proyecto funcional con múltiples tecnologías reales. - Seguridad mediante cifrado AES y RSA. - Arquitectura modular y extensible. - Documentación detallada y desarrollo iterativo.	Debilidades: - Integración compleja de herramientas externas. - Dependencia de servicios externos. - Curva de aprendizaje en gestión de servidores. - Limitaciones del plan gratuito en AWS. - No se generan token de seguridad cada sesión.
Oportunidades: - Alta demanda de plataformas colaborativas. - Evolución a producto educativo o comercial. - Posibilidad de integrar accesibilidad o gamificación.	Amenazas: - Competencia de plataformas consolidadas. - Cambios en APIs o servicios cloud. - Costes futuros de mantenimiento. -

5.4 Viabilidad del Proyecto

Desde el punto de vista **técnico**, el proyecto ha demostrado ser plenamente viable. A lo largo del desarrollo, se han implementado con éxito todas las funcionalidades previstas, utilizando herramientas profesionales y adoptando una arquitectura modular que facilita la escalabilidad, el mantenimiento y la futura evolución de la plataforma.

En cuanto a la **viabilidad económica**, si bien se planificó inicialmente utilizando servicios gratuitos (como el plan Free Tier de AWS), la experiencia real implicó ciertos **costes adicionales no previstos**, derivados del aprendizaje y uso de servicios en la nube por primera vez. El gasto total estimado se sitúa en torno a **35–40 euros**, distribuidos en los siguientes conceptos:

- **Cobro por IP elástica en AWS** mientras la instancia EC2 estaba apagada: aproximadamente **0,12 € al día + impuestos**, lo que se acumula rápidamente si no se libera la IP.
- **Error de configuración inicial** que generó un cobro puntual de aproximadamente **20 €**, por uso indebido de recursos fuera del Free Tier.
- **Compra de un dominio personalizado** en AWS para habilitar certificados SSL (HTTPS), con un coste aproximado de **14 € anuales más impuestos**.
- **Uso adicional de servicios de red y almacenamiento fuera del Free Tier**, en momentos puntuales del desarrollo y pruebas.

Estos costes, aunque moderados, son representativos de un entorno de desarrollo real, donde es importante gestionar cuidadosamente los recursos cloud para evitar gastos innecesarios. También reflejan un aprendizaje valioso en la administración de infraestructura, muy útil en contextos profesionales.

En conjunto, el proyecto es **viable tanto técnica como económicamente**, y demuestra que con una inversión relativamente baja es posible construir una plataforma funcional, segura y escalable, aplicando conocimientos adquiridos durante el ciclo formativo.

5.5 Cronograma de actividades:

Semana	Tarea planifica
Semana 1	Configuración de Unity, diseño de assets, servidor en .NET con WebSockets, base de datos.
Semana 2	Movimiento y sincronización de avatares, colisiones, pruebas con usuarios.
Semana 3	Implementación de chat de texto, integración de voz (WebRTC), comunicación por proximidad.
Semana 4	Sistema de intercambio de archivos con cifrado AES/RSA, integración con la nube, barra de progreso, notificaciones.
Semana 5	Optimización del servidor, creación de zonas interactivas, notificaciones de proximidad, reducción de latencia.
Semana 6	Pruebas de carga, pruebas de estabilidad y rendimiento, evaluación del almacenamiento en la nube.
Semana 7	Redacción de memoria, creación de la presentación, ensayo, últimos ajustes.
Semana 8	Correcciones de errores, mejoras de rendimiento, revisión de documentación.
Semana 9	Correcciones de errores, mejoras de rendimiento, revisión de documentación.

6 DESARROLLO

El desarrollo del proyecto *Oficina Virtual Interactiva* se ha estructurado en distintas fases que han evolucionado a partir del plan inicial, pero que han requerido adaptaciones constantes debido a las dificultades técnicas surgidas durante su implementación. En este apartado se describen las etapas reales del desarrollo, se comparan con la planificación original, y se exponen los principales resultados técnicos obtenidos. Asimismo, se incluyen diagramas UML relevantes para representar la arquitectura y componentes del sistema.

6.1 Fases del desarrollo real y comparación con la planificación

Durante el desarrollo del proyecto, se llevaron a cabo distintas fases organizadas por funcionalidades clave. A continuación, se destacan los principales hitos que marcaron avances importantes en el sistema:

- **Diseño inicial del entorno en Unity 2D**, creación del prototipo base y adaptación de los assets a una cuadrícula para un sistema de movimiento coherente.
- **Implementación del sistema de autenticación**, incluyendo los formularios de login, registro y recuperación, con conexión directa a una base de datos MySQL mediante una API en PHP.
- **Integración del sistema de red Mirror**, ajustando los parámetros de sincronización y corrigiendo malas prácticas heredadas de ejemplos externos, como el uso ineficiente de `FindObjectOfType` y estructuras `break` innecesarias.
- **Desarrollo del sistema de envío y recepción de archivos**, alojados en Amazon S3, con cifrado automático SSE-S3 y validación de usuarios mediante tokens de seguridad.
- **Incorporación del chat de voz con Vivox**, superando errores por incompatibilidad entre el SDK de voz y el modo de servidor de Unity, y adaptando la lógica para la gestión de salas de voz.
- **Refactorización y mejora de la interfaz gráfica**, rediseñando los paneles y menús para que se adapten a distintas resoluciones y sean más accesibles para los usuarios.

A continuación, se muestra un fragmento de código relevante correspondiente a la **sincronización de movimiento de los jugadores**, una de las partes clave en la implementación de la experiencia multijugador.

Este código permite que el movimiento y las animaciones de los jugadores se sincronicen entre todos los clientes conectados, asegurando una experiencia coherente y sin desincronización visual. Es un ejemplo de cómo se utiliza la infraestructura de **Mirror** para gestionar la sincronización de los datos entre el servidor y los clientes, utilizando las características de **SyncVar** y **Command** para mantener el estado de la animación y el movimiento del jugador en todos los clientes.

```

[SyncVar(hook = nameof(OnDirXChanged))] private float syncDirX;
[SyncVar(hook = nameof(OnDirYChanged))] private float syncDirY;
[SyncVar(hook = nameof(OnIsMovingChanged))] private bool syncIsMoving;

[Command]
void CmdUpdateAnimation(float newDirX, float newDirY, bool newIsMoving)
{
    syncDirX = newDirX;
    syncDirY = newDirY;
    syncIsMoving = newIsMoving;
}

private void OnDirXChanged(float oldValue, float newValue)
{
    if (animator != null)
        animator.SetFloat("DirX", newValue);
}

private void OnDirYChanged(float oldValue, float newValue)
{
    if (animator != null)
        animator.SetFloat("DirY", newValue);
}

private void OnIsMovingChanged(bool oldValue, bool newValue)
{
    if (animator != null)
        animator.SetBool("isMoving", newValue);
}

```

En este fragmento, se destacan varias características clave:

1. **Sincronización de la dirección y el estado de movimiento:**

Las variables `syncDirX`, `syncDirY` y `syncIsMoving` están marcadas con el atributo `[SyncVar]`, lo que significa que se sincronizan automáticamente entre el servidor y todos los clientes. Cada vez que el servidor actualiza estas variables, el valor se propaga a todos los clientes conectados.

2. **Uso de Hooks para actualización de animaciones:**

Cuando los valores de las variables `syncDirX`, `syncDirY` o `syncIsMoving` cambian, los métodos `OnDirXChanged`, `OnDirYChanged` y `OnIsMovingChanged` se invocan automáticamente. Estos métodos actualizan el estado de las animaciones del jugador

en todos los clientes, lo que asegura que todos los jugadores vean las animaciones correspondientes al movimiento del personaje.

3. Comando **CmdUpdateAnimation**:

Este método se ejecuta en el servidor y es responsable de actualizar los valores de las variables sincronizadas. Los comandos en **Mirror** permiten que el cliente envíe solicitudes al servidor para realizar cambios en el estado de los datos.

4. Coherencia de animaciones en el multijugador:

Al sincronizar el estado de las animaciones con estos **SyncVars**, se logra que todos los jugadores vean las mismas animaciones y movimientos, evitando problemas de desincronización visual que podrían surgir en un entorno multijugador.

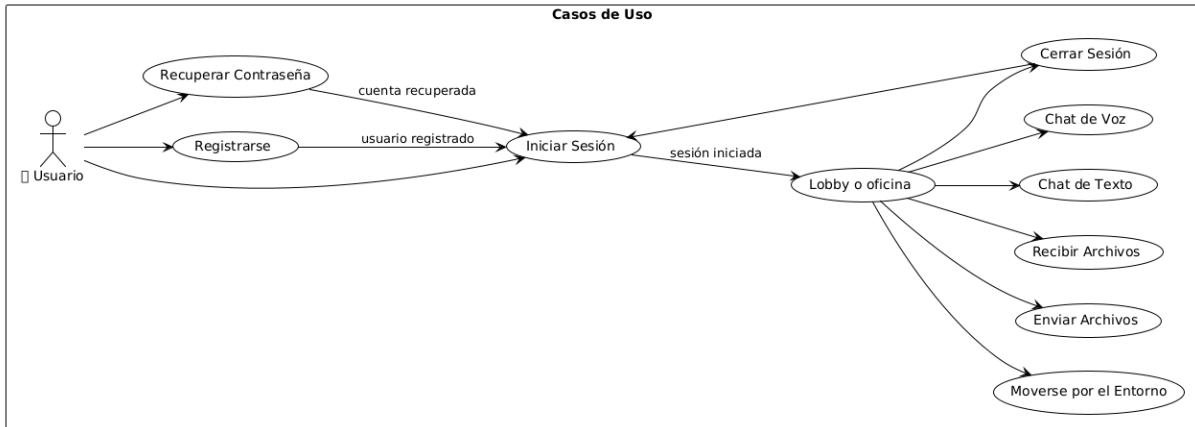
Este código es un componente esencial para garantizar que la experiencia multijugador sea fluida y coherente, especialmente en juegos donde el movimiento y las animaciones juegan un papel importante en la interacción entre los jugadores. La implementación de **Mirror** y las técnicas de sincronización de estado contribuyen a una jugabilidad más consistente y sincronizada.

6.2 Resultados obtenidos

- **Aplicación funcional multiplataforma** desarrollada con Unity.
- **Sincronización en tiempo real** mediante Mirror, con múltiples usuarios simultáneos.
- **Login y autenticación conectados a base de datos remota** con conexión segura.
- **Transferencia de archivos con cifrado automático en la nube**, utilizando roles IAM en AWS.
- **Sistema de chat de texto sincronizado con red**, funcional e integrado en la interfaz.
- **Chat de voz por Vivox** con sistema de salas y visualización de usuarios conectados.
- **Interfaz personalizable y accesible**, con paneles de control intuitivos.

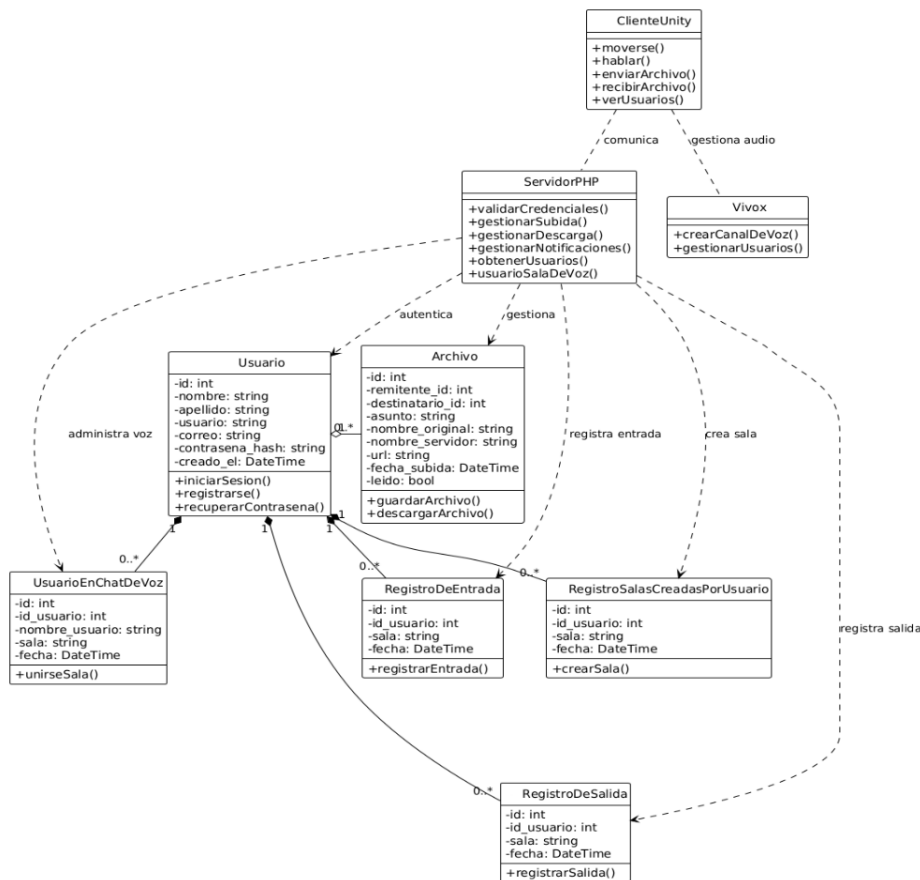
6.3 Diagramas UML

6.3.1 Diagrama de caso de uso



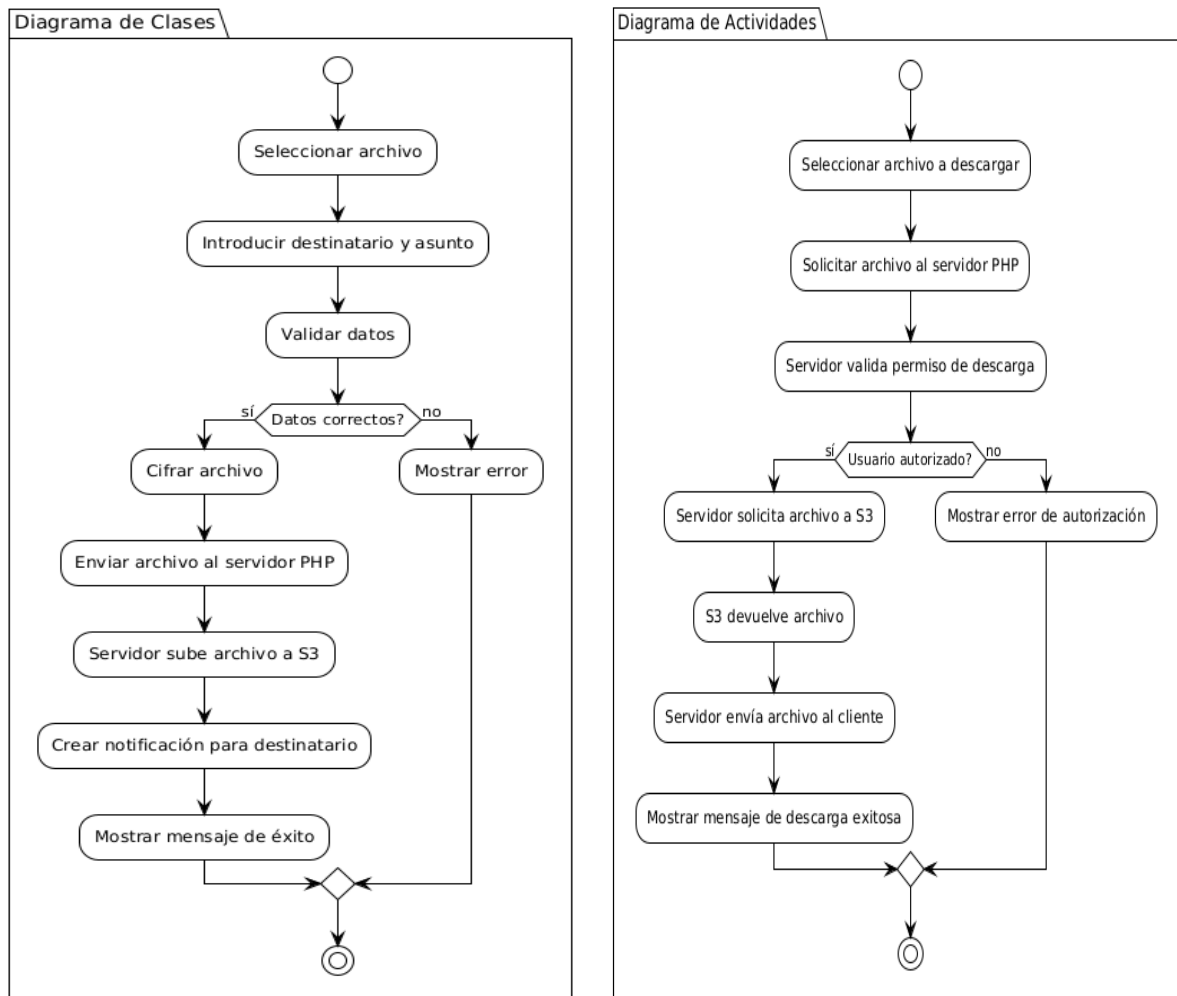
Este diagrama muestra todas las interacciones que el usuario puede realizar dentro del sistema.

6.3.2 Diagrama de clases



Se ha seleccionado este diagrama de clases debido a muestra el funcionamiento del programa, el cual un usuario normal no podría ver.

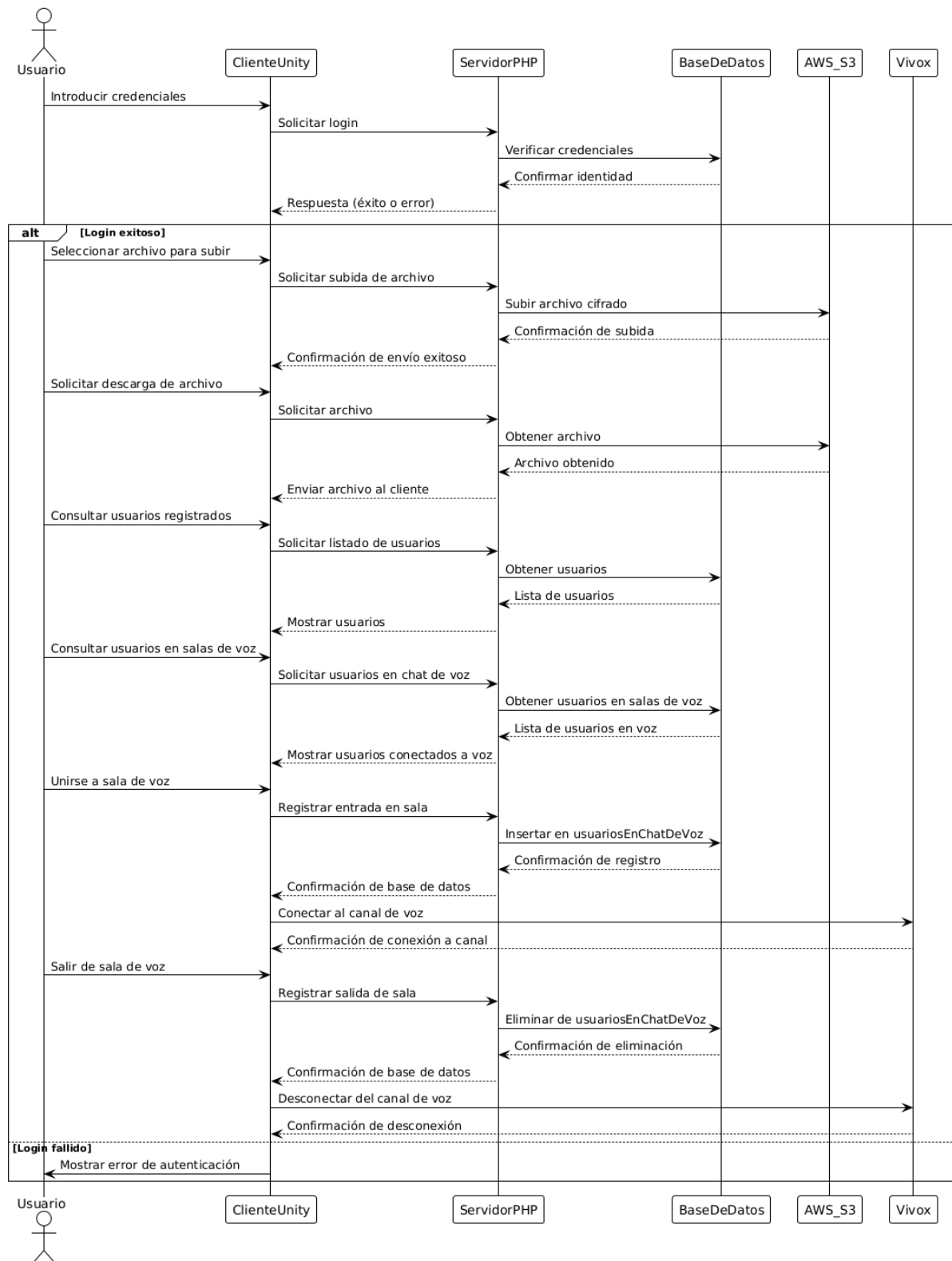
6.3.3 Diagrama de actividades



Los dos diagramas anteriores explican el sistema que permite la comunicación entre usuarios mediante el envío y recepción de archivos a través de una plataforma centralizada. Para garantizar la seguridad y la trazabilidad, se integran procesos como cifrado de archivos, control de acceso y notificaciones. Los usuarios pueden subir archivos destinados a otros usuarios, y estos pueden posteriormente descargarlos desde la nube.

Se ha mostrado los procesos de subida y descarga de archivos debido a que constituyen funcionalidades **críticas** del sistema. Estos procesos involucran una interacción compleja entre el cliente, el servidor y un servicio externo (como AWS S3). A diferencia de otras acciones más simples o locales (como iniciar sesión o visualizar usuarios), estas actividades requieren una coordinación entre múltiples componentes del sistema y garantizan la integridad y privacidad de los datos.

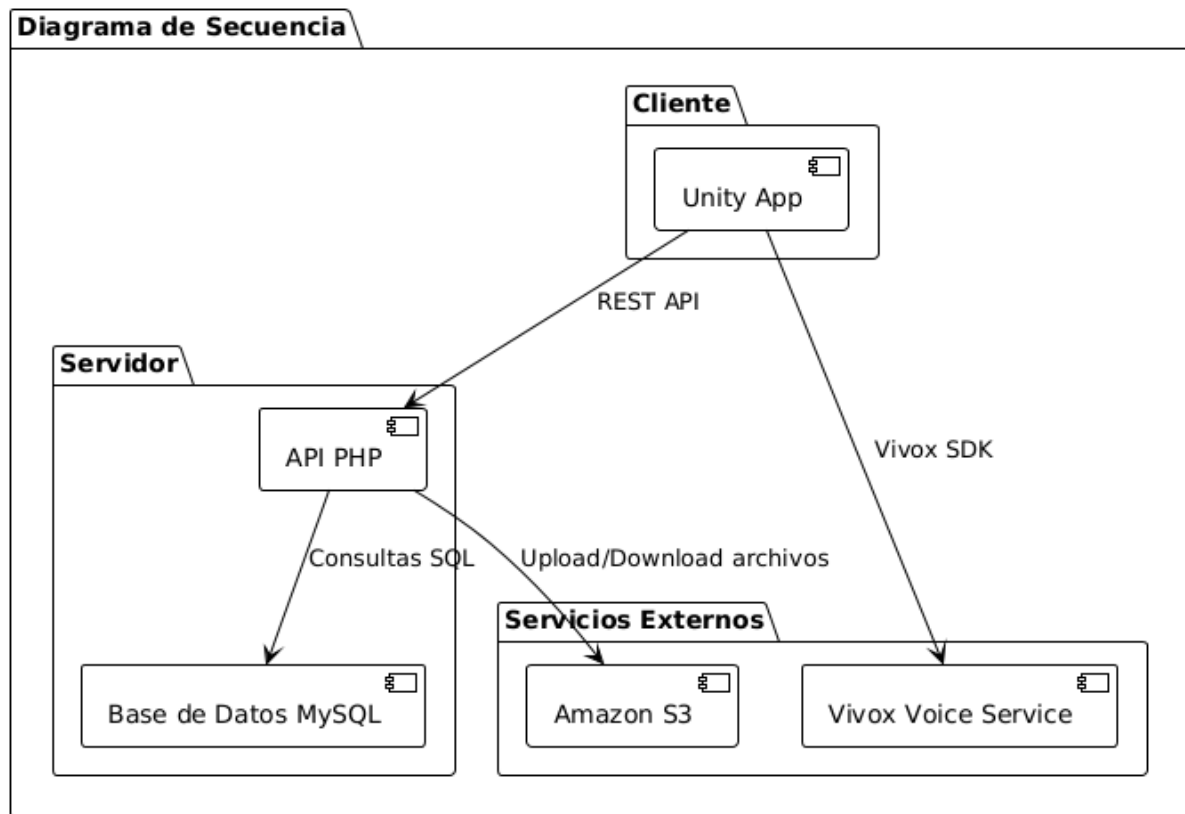
6.3.4 Diagrama de secuencia



El diagrama de secuencia representa la interacción tiempo-dependiente entre los actores y componentes del sistema cuando un usuario inicia sesión y realiza diversas acciones como subir/descargar archivos, consultar usuarios, y participar en salas de voz.

Este diagrama fue seleccionado por su valor narrativo y técnico: muestra con precisión el flujo de trabajo principal del sistema desde la perspectiva del usuario. Ilustra cómo distintos componentes interactúan en función del éxito o fallo de las acciones, y evidencia los **puntos críticos de integración** entre software cliente, servidor, base de datos y servicios externos.

6.3.5 Diagrama de componente



Este diagrama muestra los componentes que componen la aplicación, mostrando como se conectar entre sí.

7 PLAN DE PRUEBAS

El objetivo de este plan de pruebas es verificar que todas las funcionalidades clave del sistema se comportan de acuerdo con los objetivos definidos en el proyecto y cumplen con los requisitos funcionales y técnicos establecidos. Las pruebas se han dividido en pruebas funcionales, pruebas de rendimiento, pruebas de seguridad y pruebas de usabilidad.

7.1 Metodología de prueba

Las pruebas se realizaron de forma manual en distintos entornos:

- Entorno local (localhost) para pruebas de desarrollo rápido.
- Entorno de servidor (AWS EC2) para pruebas reales de red y transferencia de archivos.
- Se utilizaron dos equipos distintos para simular múltiples usuarios simultáneamente.

Cada funcionalidad fue validada tras su implementación mediante pruebas repetidas y depuración activa. Se documentaron los errores encontrados y las soluciones aplicadas.

7.2 Pruebas funcionales

Funcionalidad probada	Descripción	Resultado esperado	Resultado obtenido
Registro e inicio de sesión	Usuario se registra, luego inicia sesión con sus credenciales.	Acceso correcto o error claro en caso contrario.	Correcto
Movimiento de avatar	Usuario puede moverse en el entorno 2D.	Movimiento fluido con animaciones correctas.	Correcto
Sincronización en red (Mirror)	Varios usuarios se conectan y ven sus movimientos en tiempo real.	Avatares sincronizados sin lag excesivo.	Los avatares no se ven bien sincronizados
Envío de archivos cifrados	Usuario A envía archivo a B a través del sistema.	Archivo subido, almacenado, y recibido correctamente.	Correcto
Notificaciones de nuevos archivos	Usuario recibe aviso de que ha recibido un archivo.	Panel muestra remitente, fecha y archivo.	Correcto
Chat de texto	Usuarios escriben mensajes y estos se muestran en pantalla.	Mensaje recibido una sola vez con nombre del remitente.	Correcto tras ajustes
Chat de voz (Vivox)	Usuarios se comunican por voz en el mismo canal.	Audio claro, conexión automática al canal.	Correcto
Gestión de salas de voz	Usuario accede a diferentes salas o crea una nueva.	Conexión correcta y datos actualizados en BBDD.	Correcto
Subida/descarga desde la nube	Archivos se suben y descargan desde AWS S3.	Transferencia sin pérdida, cifrado activado.	Correcto
Descarga de archivos sin mensaje de advertencia	Se descarga un archivo y no sale un mensaje de	Descargar el archivo si ninguna advertencia	Correcto

	advertencia en el buscador de virus		
Compartir pantalla	Cuando el usuario abra el panel le dará al botón de compartir pantalla y empezara a ver su compartición y los otros usuarios también la verán.	La pantalla se comparte con los demás usuarios.	El usuario que comparte la pantalla ve que funciona bien, cuando los otros usuarios intentan conectarse a esa compartición les hecha del programa y no les permite conectarse otra vez, y en caso de que se conecten, les muestra una pantalla en negro y el programa va con lag. Para solucionarlo, hay que reiniciar el servidor.

7.3 Pruebas de rendimiento

Elemento evaluado	Criterio de prueba	Resultado
Latencia de red	Tiempo de respuesta entre acciones del cliente y reflejo en otro cliente	Aceptable (<200 ms)
Tiempo de subida de archivos	Archivo de 1.5 MB	Menos de 3 segundos
Tiempo de descarga	Archivo de 1.5 MB desde S3	Menos de 2 segundos
Carga máxima de usuarios	3 clientes simultáneos en red local y AWS	Sin caídas o errores

7.4 Pruebas de seguridad

Aspecto probado	Prueba	Resultado
Cifrado en subida de archivos	Verificación de uso de SSE-S3 en AWS S3	Confirmado
Autenticación de usuario	Pruebas con credenciales inválidas	Denegado correctamente
Protección de API	Inclusión de tokens en peticiones PHP	Aplicado correctamente
Acceso no autorizado	Acceso a rutas/archivos sin loguearse	Restringido

Certificado SSL y HTTPS	Verificar que se usa https y no http y que se disponga de certificado SSL	Correcto
-------------------------	---	----------

7.5 Pruebas de usabilidad

Se realizaron pruebas básicas con dos usuarios para verificar:

- La facilidad de uso de los paneles de login, archivos y salas.
- La accesibilidad en pantallas pequeñas (portátil).
- El feedback visual en botones y acciones.

Resultados: Los resultados obtenidos fueron los esperados; se presentaron algunos retrasos (lags), atribuibles al cuello de botella en el servidor

Se realizaron pruebas para la compartición de pantalla con diversos usuarios:

- El usuario ve su propia compartición.
- Los otros usuarios ven la compartición de la otra persona.

Resultados: El usuario logra ver su compartición , pero los otros la llegan a ver por un milisegundo antes de que le heche de la aplicación.

8 RESULTADO Y CONCLUSIONES Y VÍAS DE MEJORA

8.1 Resultados obtenidos

El proyecto ha logrado implementar con éxito la mayoría de los objetivos planteados en la fase inicial de planificación. A través del desarrollo iterativo y la validación progresiva de cada funcionalidad, se ha construido una plataforma funcional que simula un entorno de oficina virtual con interacción entre usuarios, subida y descarga de archivos, y comunicación por voz en tiempo real.

Los principales resultados obtenidos incluyen:

- Un entorno 2D interactivo donde múltiples usuarios pueden desplazarse simultáneamente con sus avatares en red, usando el motor Unity junto con el sistema Mirror para la sincronización.
- Un sistema de autenticación completo (registro, inicio de sesión y recuperación de cuenta), conectado a una base de datos MySQL remota mediante una API desarrollada en PHP.
- La posibilidad de enviar y recibir archivos entre usuarios, con almacenamiento seguro en AWS S3 y cifrado automático mediante SSE-S3.

- La incorporación de chat de texto sincronizado en tiempo real y la integración exitosa de Vivox para gestionar salas de chat de voz.
- Una interfaz de usuario funcional y adaptativa que centraliza las funciones clave del sistema (chat, notificaciones, subida de archivos, gestión de canales de voz, etc.).

A nivel técnico, también se han superado barreras relevantes, como la configuración de servidores en AWS, la gestión de certificados SSL, el diseño de seguridad básica mediante tokens, y la resolución de problemas de sincronización entre cliente y servidor.

8.2 Conclusiones

El sistema desarrollado ha demostrado ser funcional y técnicamente viable, utilizando servicios de bajo coste como EC2 t2.micro y Amazon S3 con cifrado SSE-S3. La aplicación permite la conexión simultánea de varios usuarios con una estabilidad aceptable, integrando correctamente las funcionalidades clave: sincronización en red mediante Mirror, chat de texto, chat de voz (Vivox) y transferencia de archivos cifrados.

Durante el desarrollo se identificaron limitaciones propias del uso de recursos gratuitos, especialmente en términos de rendimiento. La instancia EC2 t2.micro no es adecuada para funcionalidades de alta demanda, como la compartición de pantalla, que no pudo implementarse de forma estable. Esta función genera un cuello de botella en el servidor, lo que provoca caídas en el rendimiento general de la aplicación. Por tanto, se concluye que sería necesario escalar a instancias de mayor capacidad para soportar este tipo de funcionalidades en entornos reales.

Para garantizar el uso de HTTPS y certificados SSL válidos, se adquirió un dominio personalizado, con un coste estimado de 14 €. En total, los costes asociados al uso de servicios en la nube y a la configuración del sistema ascienden a unos 35–40 €, lo cual representa una inversión reducida para una arquitectura funcional y segura.

El sistema presenta una arquitectura modular y escalable, lo que facilita su ampliación e integración con nuevos servicios. Esto lo hace apto para su evolución en escenarios como enseñanza remota, asistencia técnica o trabajo colaborativo en línea.

8.3 Vías de mejora

Aunque el proyecto ha alcanzado un nivel funcional sólido, existen múltiples áreas donde se podrían implementar mejoras y ampliaciones en el futuro:

- **Optimización del rendimiento gráfico y de red:** ajustar aún más la sincronización entre clientes para eliminar el pequeño lag que aparece en movimientos remotos, especialmente en conexiones con latencia alta.
- **Mejora visual y estética:** rediseñar paneles, fondos y animaciones para mejorar la experiencia de usuario y ofrecer un entorno más atractivo visualmente.
- **Ampliación del sistema de permisos y roles:** permitir que algunos usuarios tengan funciones de administración (expulsar, crear reuniones privadas, etc.).

- **Notificaciones en tiempo real:** sustituir las consultas periódicas a la base de datos por un sistema basado en eventos (websockets o notificaciones push).
- **Historial de conversación y chat persistente:** guardar las conversaciones para que estén disponibles tras cerrar sesión.
- **Compatibilidad multiplataforma:** crear versiones para Android, iOS o ejecutables web que faciliten el acceso desde distintos dispositivos.
- **Auditoría y registro de actividad:** almacenar información adicional sobre conexiones, transferencias y acciones relevantes para entornos profesionales.
- **Sistema de token dinámico para mayor seguridad:** implementar un sistema de autenticación con tokens temporales o JWT (JSON Web Tokens), más seguro que el token estático actual.
- **Recuperación avanzada de cuenta:** añadir una opción para recuperar el acceso si se olvida el nombre de usuario, mediante el envío de un token temporal al correo electrónico asociado.
- **Funcionalidad de compartir pantalla:** explorar la integración de un sistema de compartición de pantalla dentro de la oficina virtual, para permitir presentaciones o asistencia remota entre usuarios.

Estas mejoras no solo enriquecerían la experiencia del usuario, sino que también posicionarían la plataforma como una solución más robusta y escalable para entornos reales de teletrabajo o colaboración virtual.

9 ANEXOS

9.1 Documentos de planificación

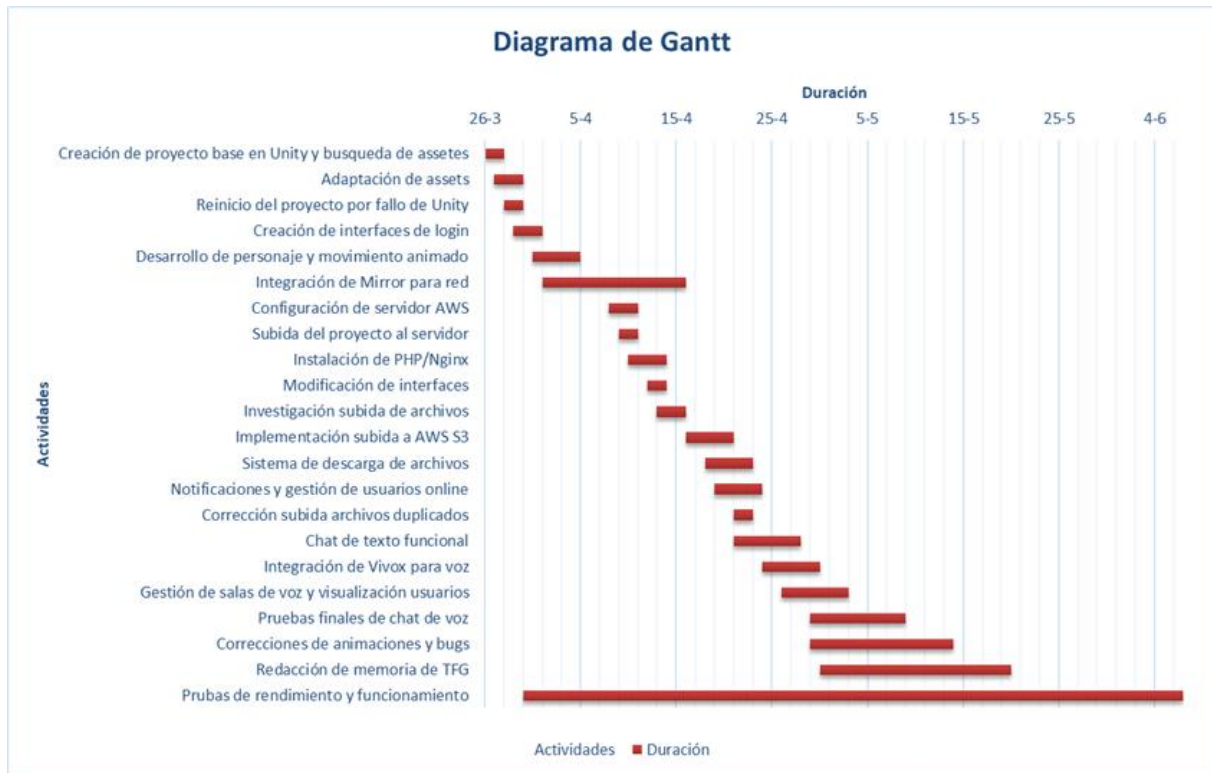
9.1.1 Plan de hitos

El siguiente plan de hitos detalla los principales puntos de control establecidos durante el desarrollo del proyecto, asegurando el progreso y cumplimiento de los objetivos planificados.

Hito	Descripción
Entorno base configurado	Unity y servidor operativo, base de datos inicial
Sincronización de avatares completada	Avatares moviéndose en tiempo real con Mirror
Comunicación en tiempo real estable	Chat de texto y voz funcional entre usuarios
Sistema de archivos operativo	Subida/descarga de archivos cifrados
Optimización de la plataforma	Reducción de latencia
Superadas pruebas de carga	Plataforma soporta múltiples usuarios simultáneos

El cumplimiento de estos hitos ha permitido garantizar un desarrollo progresivo y ordenado del proyecto, facilitando la entrega de una solución estable, funcional y documentada.

9.1.2 Diagramas de Gantt



9.1.3 Cronograma de actividades:

Fecha	Actividad Detallada
26/03/25	Inicio del proyecto: búsqueda de assets para el entorno y creación del proyecto base en Unity.
27/03/25	Adaptación de assets al sistema de cuadrícula de Unity. Búsqueda de sprites para el personaje principal y creación de la escena 'Lobby'.
28/03/25	Reinicio completo del proyecto debido a errores críticos en la última versión de Unity. Se opta por una versión estable y se redibuja el mapa.
29/03/25	Creación de la segunda escena con los elementos necesarios. Desarrollo de interfaces de autenticación: login, registro y recuperación de cuenta. Primeros scripts

	funcionales para la navegación entre paneles.
30/03/25	Día libre.
31/03/25	Sesión de tutoría y corrección de errores. Implementación del personaje jugable, armado manual de animaciones en 8 direcciones y programación de su movimiento con animaciones asociadas.
01/04/25 – 07/04/25	Integración de Mirror para la comunicación en red. Se refactoriza código heredado de un tutorial por malas prácticas (break en if, uso excesivo de FindObjectOfType). Se crean scripts propios para mejorar rendimiento y estructura. Problemas con sincronización de animaciones, cambio de escenas y conexión cliente-servidor ocupan más de 5 días de trabajo intensivo.
08/04/25	Configuración inicial del servidor en AWS. Se experimentan múltiples errores por falta de experiencia y por seleccionar regiones incorrectas, lo que lleva a repetir el proceso varias veces.
09/04/25	Subida del proyecto al servidor utilizando Putty y WinSCP. Investigación y prueba de nuevas opciones de exportación en Unity (dedicated server vs. server build) debido a cambios recientes en la herramienta server build fue descontinuada.
10/04/25	Configuración de la base de datos MySQL en AWS. Creación y migración de tablas. Inicio en el uso de PHP para construir la API de comunicación con el servidor.
11/04/25	Intento fallido de instalación de Apache + PHP. Se cambia a Nginx + PHP, obteniendo resultados satisfactorios. El servidor logra ejecutar correctamente los scripts en PHP.
12/04/25	Modificación de la interfaz de login para hacerla fija. Eliminación del HUD de red de Unity para una conexión automática. Creación de tabla en la base de datos para gestionar archivos y su integración en la interfaz.
13/04/25	Investigación de soluciones para subida de archivos. Primer intento fallido de implementación de chat de texto.
14/04/25	Segundo intento fallido de desarrollo del chat de texto.
15/04/25	Día de descanso.
16/04/25	Redirección del enfoque hacia la transferencia de archivos. Configuración del

	arranque automático del servidor junto con la aplicación. Inclusión de tokens de seguridad en los scripts PHP. Creación de sistema en la nube (AWS S3) para almacenamiento cifrado de archivos.
17/04/25	Solución de errores en el servidor. Implementación exitosa de subida a S3 sin exponer credenciales mediante roles IAM. Configuración del bucket para usar cifrado SSE-S3, protegiendo la confidencialidad de los datos.
18/04/25	Corrección de fallos en el servidor que provocaban el borrado accidental de archivos tras su subida.
19/04/25	Desarrollo del sistema de descarga de archivos. Diseño del gestor de notificaciones y comunicación con el servidor usando tokens seguros.
20/04/25	Creación del panel para visualizar usuarios registrados. Primeros pasos para vincular chat de texto a Mirror. Corrección de errores en interfaces UI y base de datos.
21/04/25	Corrección de sobreescritura de archivos subidos. Uso de identificador único y almacenamiento del nombre original. Desarrollo del sistema de bloqueo de movimiento al abrir menús y nuevo intento de implementación funcional del chat de texto.
22/04/25	Redacción del resumen ejecutivo. Resolución de errores de estabilidad en el servidor que causaban apagados automáticos.
23/04/25	Generación de build para pruebas finales: login multiusuario, subida y descarga de archivos, validación del sistema de chat de texto. Inicio de planificación para el chat de voz.
24/04/25	Integración de Photon Voice fallida. Unity no reconoce componentes clave. Tras 8 horas sin éxito, se decide cambiar de herramienta.
25/04/25	Elección de Vivox como sistema de chat de voz. Registro en Unity Cloud y vinculación del proyecto al servicio. Preparación del entorno para su integración.
26/04/25	Implementación de Vivox y debugging intensivo (8h). Tras múltiples pruebas fallidas, se detecta que el problema era el modo de compilación 'Dedicated Server'. Al

	cambiar a modo PC/Windows, el sistema funciona correctamente. Se prueba canal de eco con éxito.
27/04/25	Diseño de paneles de control de voz. Gestión de salas privadas y públicas. Creación de tablas en la base de datos para monitorizar usuarios en cada sala. Optimización de scripts de comunicación cliente-servidor.
28/04/25	Jornada de descanso por apagón eléctrico.
29/04/25	Pruebas en vivo del chat de voz con dos dispositivos. Detección de errores menores en animaciones remotas.
30/04/25	Redacción de la memoria del proyecto.
01/05/25	Redacción de la memoria del proyecto.
02/05/25	Redacción de la memoria del proyecto y corrección de errores de lag en las animaciones de los personajes.
03/05/25	Alquilar un dominio (14\$ + impuesto (3\$) y asignárselo a mi IP para poder tener certificados SSL y poder descargar archivos sin que salga la notificación de archivos no seguros y poder usar "https". Cambiar la dirección en todo el código para que se dirija al dominio y no a la IP. Con esto podemos tener acceso al https algo que no teníamos antes y ahora podemos tener certificados SSL para la seguridad de nuestros datos al ser transmitidos.
04/05/25	Intento de solucionar lag en las animaciones de los personajes al moverse. Modificar panel notificaciones para que sea más ordenado y los datos se vean mejor. Modificar script del gestor de notificaciones para que haga una consulta cada 30 segundos en vez de 10 para reducir el tráfico.
05/05/25	Intentar solucionar Lag
06/05/25	Intentar solucionar Lag
07/05/25	Crear un botón en la escena de Login que permita salir de la app. Crear Script para que el botón funcione. Crear una imagen para la escena del Login. Corrección de fallos en el servidor debido al dominio.
08/05/25	Intentar solucionar Lag
09/05/25	Tutoría
10/05/25	Intentar solucionar Lag minimizando los post al servidor
11/05/25	Memoria del proyecto
12/05/25	Descanso

13/05/25	Descanso
14/05/25	Desarrollar los paneles correspondientes a la funcionalidad de compartir pantalla. Integrar nuevos botones en el panel principal de control (donde se agrupan los botones de acciones). Además, implementar la lógica necesaria para que, al hacer clic en el botón correspondiente, se active la visualización del panel de compartir pantalla.
15/05/25	Investigar y consultar documentación oficial, foros especializados y guías de uso de la herramienta Mirror para implementar correctamente la funcionalidad de compartir pantalla. Basarse en buenas prácticas recomendadas por la comunidad y ejemplos técnicos existentes para garantizar una integración eficaz.
16/05/25	Completar la implementación del código necesario para habilitar el sistema de compartir pantalla. Integrar dicha funcionalidad con el panel de la interfaz de usuario, asegurando la correcta vinculación del código. Asignar a los botones del panel las acciones correspondientes para activar, controlar y detener el uso compartido de pantalla.
17/05/25	Se generó una nueva build del proyecto destinada al servidor, con el objetivo de actualizar la versión existente debido a recientes modificaciones en la configuración de red. Tras la implementación de estos cambios, se procedió a realizar pruebas de la funcionalidad de compartir pantalla. La funcionalidad fue verificada correctamente desde el lado del usuario que inicia la compartición, confirmando su correcto funcionamiento en ese contexto.
18/05/25	Durante la prueba de la funcionalidad de compartición de pantalla en un entorno con varios usuarios simultáneos, se detectaron errores críticos en la aplicación. Al iniciar la compartición, los usuarios receptores lograban visualizar la pantalla durante apenas un milisegundo antes de que la aplicación se cerrara inesperadamente (crasheo). Tras ser expulsados, los usuarios no pudieron volver a acceder con normalidad: al intentar reconectarse, la pantalla permanecía en negro y la aplicación no respondía adecuadamente.

	Por otro lado, el usuario que inició la compartición no fue desconectado, pero experimentó un retraso muy acusado (lag severo) al mover su avatar dentro del entorno, aunque continuó visualizando su pantalla compartida. Para mitigar el problema, fue necesario reiniciar el servidor. Una vez realizado este procedimiento, los usuarios pudieron volver a acceder a la aplicación con normalidad.
19/05/25	Intento de solucionar la compartición de pantalla. Modificación de varios paneles.
20/05/25	Intento de solucionar la compartición de pantalla. Limpieza de código.
21/05/25	Intento de solucionar la compartición de pantalla. Ajustes en la secuencia del proyecto a la hora de su funcionamiento.
22/05/25	Intento de solucionar la compartición de pantalla. Preparación de la presentación del proyecto.
23/05/25	Intento de solucionar la compartición de pantalla. Preparación de la presentación del proyecto.
24/05/25	Revertir el proyecto a una versión antigua y dejar deshabilitado la compartición de pantalla.
25/05/25	Se realizaron ajustes en los paneles de la aplicación para lograr una disposición más simétrica y visualmente equilibrada. Además, se corrigió el comportamiento de los componentes internos, que previamente se desplazaban o desalineaban al cambiar el tamaño de la ventana o la resolución del monitor. Con esta modificación, los elementos permanecen estables y correctamente posicionados independientemente de la configuración de pantalla del usuario.
26/05/25	Tutoría.
27/05/25	Se realizaron modificaciones en el proceso de cambio de escena, incorporando ahora un mensaje de confirmación que solicita al usuario confirmar si desea proceder con el cambio. Además, se ajustó la lógica del código responsable del cambio de escena. Anteriormente, la ejecución dependía de la condición if(isServer) , sin embargo, en esta implementación no es viable utilizar esta condición, ya que primero debe ejecutarse localmente en el cliente y posteriormente

	solicitar autorización al servidor para llevar a cabo la acción. Para lograr esto, se utilizó el atributo [Command] de Mirror, vinculándolo al componente del personaje, el cual posee ciertos privilegios administrativos que permiten gestionar esta autorización de forma segura y controlada.
28/05/25	Limpieza de código.
29/05/25	Modificación de los paneles.
30/05/25	Pruebas del programa.
31/05/25	Pruebas del programa con varios usuarios
01/06/25	Corrección de errores.
02/06/25	Pruebas del rendimiento.

9.2 Manual de uso

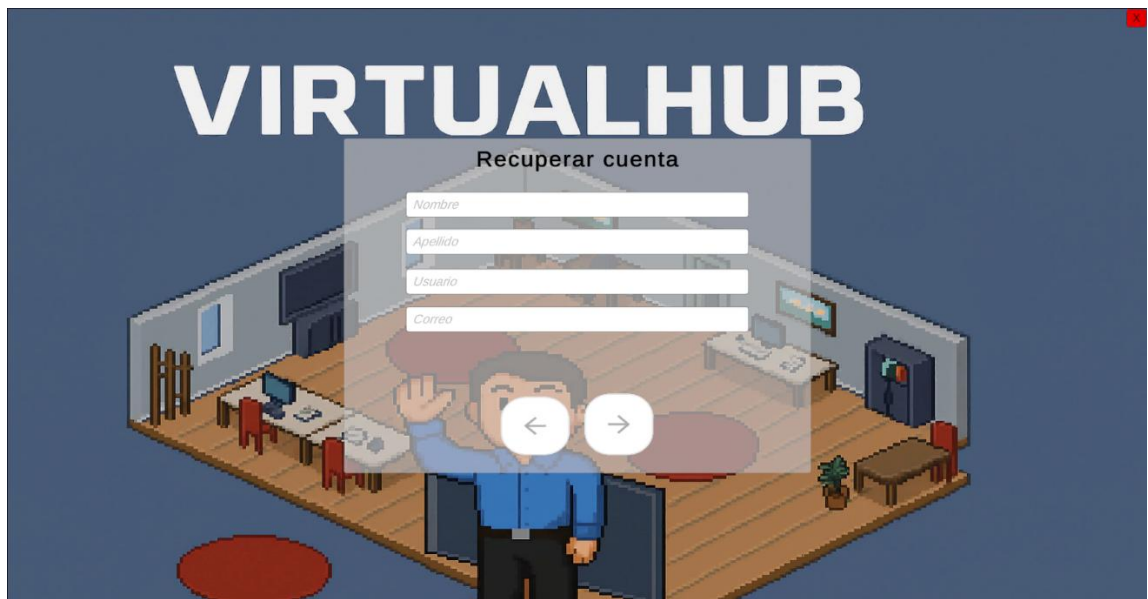
9.2.1 Acceso al programa

Al ejecutar el archivo OficinaVirtual.exe, se abrirá la interfaz de inicio de sesión. El usuario tiene tres opciones iniciales:

- **Iniciar sesión:** Si ya está registrado, puede introducir sus credenciales.



- **Registrarse:** En caso de no tener cuenta, podrá crear una nueva desde la opción de registro. **El nombre de usuario no acepta números.**
- **Recuperar contraseña:** Si ha olvidado su contraseña, puede recuperarla mediante la opción correspondiente.



Nota: Actualmente, en caso de haber olvidado el nombre de usuario, no se dispone de un mecanismo de recuperación. Se recomienda proceder con la creación de una nueva cuenta. Esta funcionalidad está prevista para futuras actualizaciones del sistema.

Nota: Para salir de la aplicación, debe pulsarse el icono de cierre (X) ubicado en la esquina superior derecha de la ventana

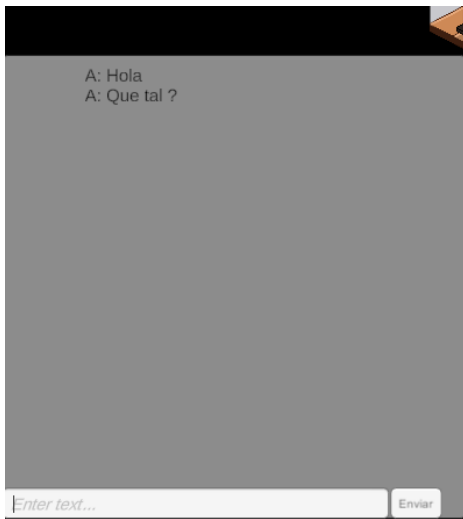
9.2.2 Entorno principal

Una vez iniciada la sesión correctamente, el usuario accederá al **lobby**, donde se encontrará su avatar en un espacio común compartido con otros usuarios.



Interfaz de usuario (UI)

- **Chat de texto (parte inferior izquierda):** Permite comunicarse en tiempo real con otros usuarios conectados.



- **Chat de voz (parte inferior derecha):** Contiene los botones para:

- Silenciar micrófono.
- Ensordecer audio recibido.
- Desconectarse del canal de voz.



- **Menú lateral derecho:** Contiene varios botones para acceder a funciones adicionales.

9.2.3 Funciones del menú lateral

1. **Enviar archivo (icono de nube con flecha):**

Permite mandar un archivo a otro usuario:

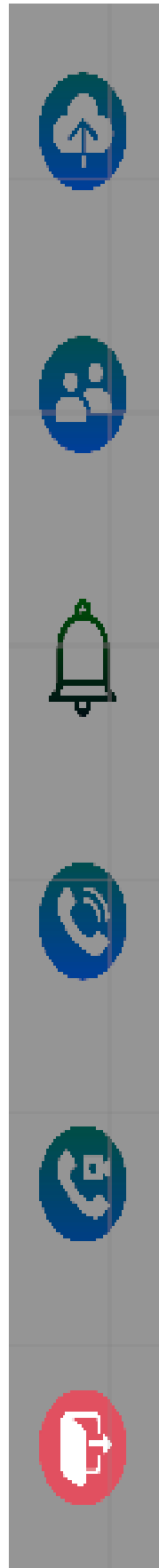
- Se solicita el nombre del destinatario.
- Introducir el asunto del archivo.
- Seleccionar el archivo desde el dispositivo.
- Al pulsar el botón de envío, se mostrará un mensaje de éxito o error.



2. **Ver usuarios registrados (icono de personas):**

Muestra una lista de todos los usuarios registrados en la oficina virtual.

- Nota: Al pulsar en el botón hay que esperar unos segundos antes de que aparezcan los datos.
- Por ahora no muestra si están en línea o no (funcionalidad prevista en futuras versiones).
- Es útil para identificar el nombre exacto de otros usuarios, especialmente para enviar archivos o identificar quién está hablando.





3. Notificaciones (icono de campana):

Muestra los archivos que el usuario ha recibido:

- Indica el nombre del remitente y el asunto.
- Se incluye un botón para descargar el archivo.
- Al pulsar este botón, se abrirá el buscador y se descargará dicho archivo.



4. Chat de voz (icono de teléfono)

Desde este panel se puede:

- Unirse a salas públicas como "Lobby" o "Oficina".
- Crear una sala privada introduciendo un nombre personalizado.
- Otros usuarios pueden unirse a esa sala privada introduciendo el mismo nombre.
- Para desconectarse, se utilizan los botones de control en la interfaz de chat de voz.



Nota: Tarda 10 segundos en mostrar el cambio en el panel, pero el cambio en la llamada tarda 3 – 5 segundos.



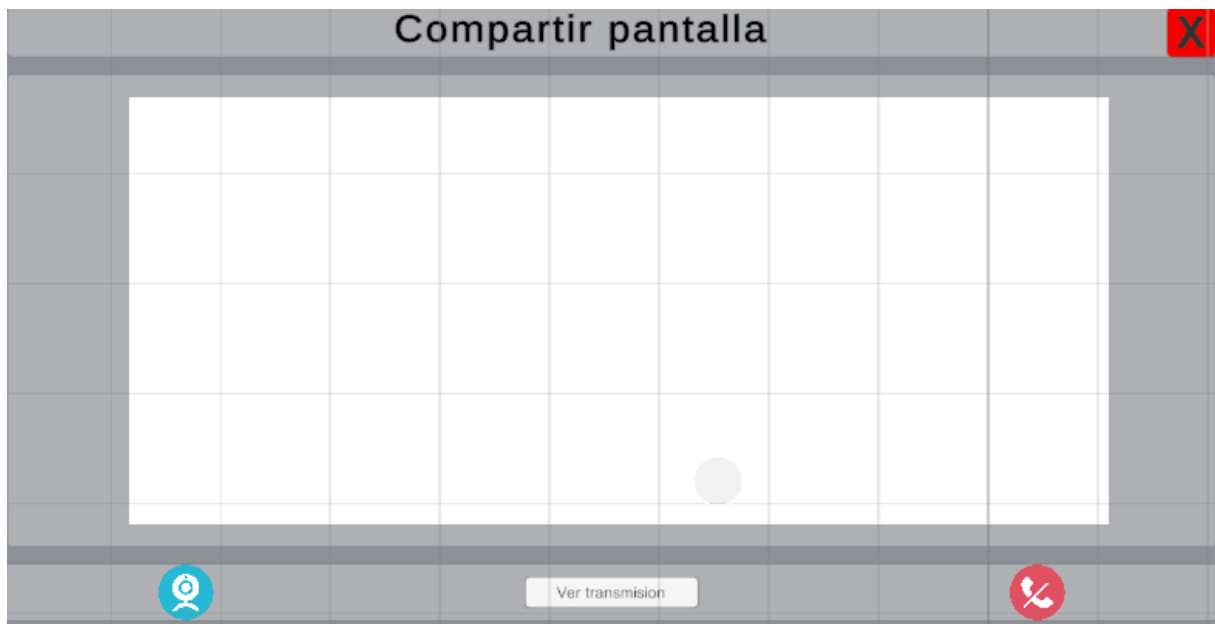
5. Compartir pantalla (botón con teléfono y cámara)

Nota: La compartición de pantalla esta **deshabilitada** pero el panel permanece activo. El motivo se explica en el plan de pruebas [7.2](#) .

- Al pulsar el botón se abre el panel
- Contiene 3 botones en la zona inferior
- Primer botón sirve para compartir
- Segundo botón para unirse a una transmisión
- Tercer botón para salir de la transmisión.



- Botón X superior sirve para cerrar el panel.



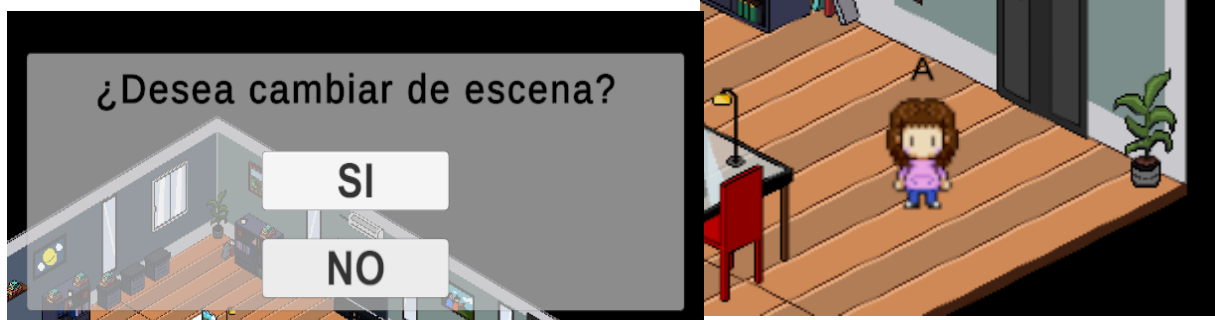
6. Cerrar sesión (botón rojo con icono de puerta):

- Cierra la sesión del usuario.
- Redirige a la pantalla de inicio de sesión (login).



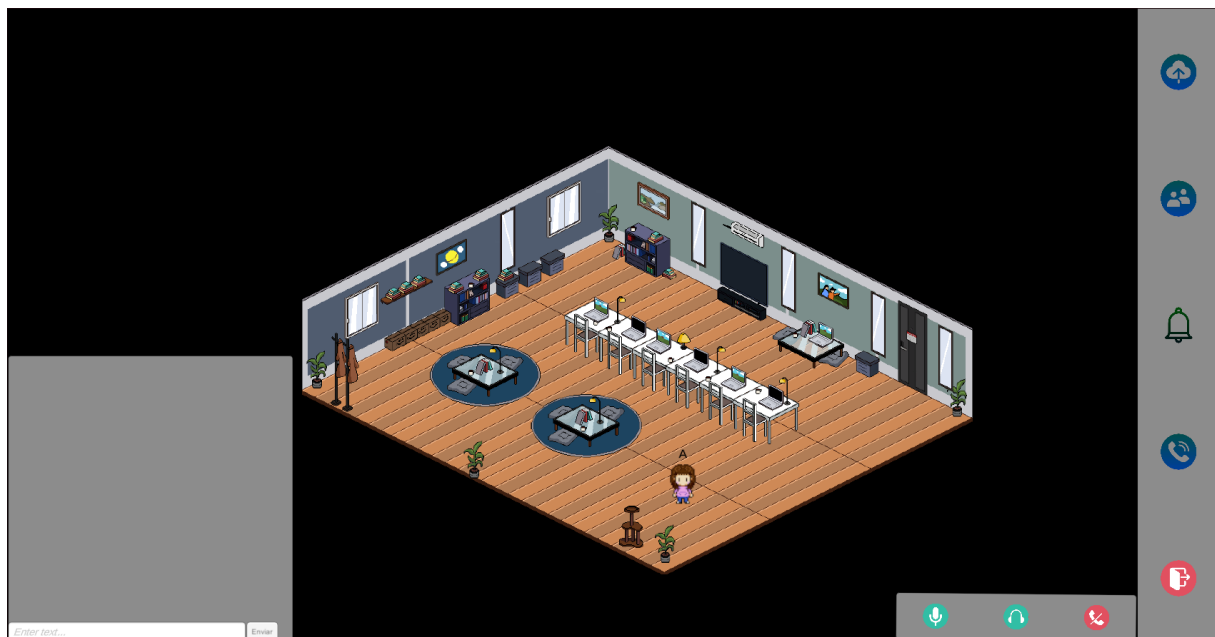
9.2.4 Navegación entre escenas

Si el usuario dirige su avatar hacia la puerta situada en el lobby, y se aproxima lo suficiente a ella, se activará el cambio de escena, aparecerá un mensaje que nos preguntará si queremos cambiar de escena, si le damos a “Si” cambiaremos de escena en el caso de darle al “No” seguiremos en la escena actual.



- Esta nueva escena mantiene la misma interfaz de usuario que el lobby.

- Todos los botones y funcionalidades son exactamente iguales.
- Si se quiere volver a la escena anterior bastara con atravesar la puerta en la nueva escena.



Nota: Cada chat de texto es independiente y está asociado a una escena específica; por tanto, no es posible mantener comunicación a través del mismo canal de chat entre distintas escenas. Esta restricción no aplica al chat de voz, el cual permite la comunicación continua entre escenas.

10 BIBLIOGRAFÍA

- Assets: Unity Low Poly Office Pack(<https://assetstore.unity.com/packages/3d/characters/low-poly-office-pack-characters-props-119386>)
- Assets: Píxel Office (<https://2dpig.itch.io/pixel-office>)
- Assets salas: (<https://bongseng.itch.io/isometric-apartment-asset-pack>)
- Assets personaje: (<https://bossnelnel.itch.io/8-direction-top-down-character-sprites>)
- Assets cocina: (<https://kabukidanshi.itch.io/isometric-pixel-art-home-interior-pack>)
- Explorador de archivos: Unity Simple File Browser (<https://github.com/yasirkula/UnitySimpleFileBrowser>)
- Iconos: Sumberrejeky en Flaticon (<https://www.flaticon.es/autores/sumberrejeky>)
- Guía de Vivox: (<https://docs.unity.com/ugs/en-us/manual/vivox-unity/manual/Unity/vivox-unity-first-steps>)
- Mirror: <https://www.youtube.com/@HomeSliceGame>
- Subir proyecto al servidor: https://www.youtube.com/watch?v=JPAhpxyusCM&ab_channel=SolidJuho
- Imagen del Login: ChatGPT.
- Diagramas: <https://planttext.com/>